

1. PROIECTAREA SISTEMULUI INTEGRAT DE MANIPULARE AUTONOMĂ ȘI INTERFAȚĂ AR

Capitolul 1 prezintă etapa de proiectare a sistemului integrat propus în această lucrare. Înainte de implementarea fizică și software, este necesară definirea clară a componentelor utilizate, a restricțiilor de proiectare, a fluxurilor de date și a rolului fiecărui subsistem. Sistemul urmărit combină o platformă mobilă autonomă existentă, un braț robotic de manipulare OMX-F, un task de pick and place proiectat cu Movelt2 Task Constructor și o interfață de vizualizare și control prin Unity, cu extensie către realitate augmentată prin Meta Quest.

În acest capitol nu se urmărește doar prezentarea unor tehnologii separate, ci modul în care acestea pot fi puse împreună într-o arhitectură coerentă. O platformă mobilă, un braț robotic, un framework de planificare a mișcării și o aplicație AR pot funcționa individual, însă valoarea proiectului apare în momentul în care ele sunt integrate într-un scenariu comun. Din acest motiv, proiectarea pornește de la analiza brațului robotic, continuă cu framework-urile software necesare și ajunge la definirea arhitecturii complete.

Contribuțiile proprii din acest capitol sunt legate de alegerea și justificarea soluției de integrare, analiza funcțională a sistemului, proiectarea amplasamentului brațului și a coșului, definirea task-ului de pick and place, proiectarea bridge-ului ROS2-Unity și arhitectura interfeței AR. Platforma mobilă și brațul robotic sunt resurse existente, dar modul în care ele sunt combinate, configurate și utilizate în scenariul propus reprezintă partea de proiectare a lucrării.

În redactarea capitolului sunt utilizate atât sursele deja introduse în partea de introducere, cât și surse suplimentare necesare pentru detaliile tehnice ale Capitolului 1. Pentru a păstra continuitatea bibliografiei, sursele noi introduse în acest capitol pornesc de la numărul [29]. Sursele deja folosite anterior își păstrează numerotarea, astfel încât să nu fie dublate în bibliografie.

1.1 Platforma mobilă autonomă Xplorer-A - prezentarea platformei de lucru existente

Platforma mobilă autonomă Xplorer-A reprezintă baza de lucru pe care se dezvoltă integrarea propusă în această lucrare. Este important de precizat încă de la început că platforma mobilă nu este proiectată de la zero în cadrul prezentului proiect. Aceasta este un sistem existent, dezvoltat anterior în cadrul FIIR-UNSTPB, iar în lucrarea de față este utilizată ca suport pentru integrarea brațului robotic OMX-F, pentru scenariul de pick and place și pentru interfața ROS2-Unity/AR. Prin urmare, rolul acestui subcapitol este de a documenta platforma disponibilă, arhitectura ei hardware și software și motivele pentru care ea este potrivită ca bază de integrare.

Descrierea platformei se bazează pe articolul „Lightweight Semantic-Aware Route Planning on Edge Hardware for Indoor Mobile Robots: Monocular Camera-2D LiDAR Fusion with Penalty-Weighted Nav2 Route Server Replanning”, în care sunt prezentate platformele Xplorer utilizate pentru cercetare în navigație autonomă indoor, inclusiv Xplorer-A [25]. Articolul are relevanță directă pentru

această lucrare deoarece descrie aceeași familie de roboți mobili, aceeași orientare către ROS2 Jazzy și Nav2 și aceeași idee de rulare pe hardware edge cu resurse limitate. În acest proiect, accentul nu este pus pe rutarea semantică în sine, ci pe extinderea platformei mobile prin manipulare robotică și interfață de realitate augmentată.

Xplorer-A este o platformă mobilă diferențială cu patru roți, proiectată pentru navigație autonomă în medii interioare. Conform articolului, platformele Xplorer au fost dezvoltate pentru cercetare și educație, fiind construite în jurul unei arhitecturi ROS2 complete, cu senzori de localizare, percepție și control al deplasării [25]. Acest lucru este important pentru lucrarea de față, deoarece integrarea brațului nu pornește de la un robot mobil simplu, comandat manual, ci de la o platformă care are deja un stack de navigare autonomă.

Din punct de vedere mecanic, platforma este de tip 4WD diferențial. Această configurație presupune controlul diferențial al roților din stânga și din dreapta, astfel încât robotul se poate deplasa înainte, înapoi și se poate roti prin diferență de viteză între cele două părți. Pentru un sistem de laborator, această soluție este robustă și relativ ușor de controlat. În același timp, integrarea unui braț robotic pe o astfel de platformă trebuie făcută cu atenție, deoarece masa suplimentară și poziția brațului pot influența stabilitatea, distribuția greutății și comportamentul în timpul deplasării.

Din punct de vedere al unității de calcul, articolul descrie utilizarea unui Raspberry Pi 5 cu 16 GB RAM, rulând Ubuntu 24.04 și ROS2 Jazzy [25]. Alegerea unui astfel de hardware este relevantă deoarece arată că platforma este gândită pentru execuție pe edge hardware, adică pe un calculator compact, cu consum redus și fără resurse de calcul comparabile cu o stație desktop. În articol, această constrângere este tratată în contextul rulării detecției YOLO și a rutării semantice. În lucrarea de față, aceeași constrângere trebuie luată în calcul pentru integrarea brațului, a nodurilor ROS2 suplimentare și a bridge-ului către Unity.

Tabelul 1.1.1 prezintă principalele elemente hardware ale platformei Xplorer-A, așa cum sunt relevante pentru proiectul de față.

Tabelul 1.1.1 - Componente hardware relevante ale platformei Xplorer-A

Componentă	Rol în platformă	Relevanță pentru proiectul curent
Șasiu 4WD diferențial	Bază mecanică pentru deplasare indoor	Suport pentru integrarea brațului OMX-F
Raspberry Pi 5	Unitate de calcul edge	Rulează ROS2, Nav2 și nodurile de integrare
LiDAR STL-19P/LD19	Scanare 2D 360° pentru navigare	Localizare, costmap și evitare obstacole
Cameră monoculară Logitech C920	Percepție vizuală	Utilă pentru extensii de percepție și documentare
IMU BNO055	Măsurare orientare/accelerații	Intrare pentru fuziune senzorială și estimare stare
Encodere roți	Feedback de mișcare	Odometrie pentru navigare
Controlere RoboClaw	Control motoare DC	Execuția comenzilor de deplasare
Baterie 12 V Li-ion	Alimentare platformă	Trebuie analizată împreună cu alimentarea brațului

Senzorul LiDAR este unul dintre elementele principale pentru navigația autonomă. În articol, platforma folosește un LiDAR 2D STL-19P/LD19 cu câmp vizual de 360°, aproximativ 503 raze și frecvență de 10 Hz [25]. Acest senzor permite robotului să observe obstacolele din jur și să construiască informația

necesară pentru localizare și evitare de obstacole. În contextul proiectului de față, LiDAR-ul rămâne important deoarece montarea brațului și a coșului nu trebuie să blocheze câmpul de scanare sau să introducă obstacole permanente neprevăzute în jurul robotului.

Camera monoculară este descrisă în articol ca parte a sistemului de percepție folosit pentru detecția obiectelor în rutarea semantică [25]. În lucrarea de față, camera nu este elementul central al task-ului de manipulare, deoarece poziția obiectului poate fi definită inițial prin parametri cunoscuți. Totuși, prezența camerei este importantă pentru dezvoltări ulterioare, cum ar fi detectarea obiectului de manipulat sau transmiterea imaginii către interfața Unity. Astfel, platforma oferă deja o bază senzorială care poate fi extinsă.

IMU-ul BNO055 și encodelele roților contribuie la estimarea mișcării platformei. Într-un sistem mobil, odometria pură poate acumula erori, iar fuziunea cu date inerțiale ajută la obținerea unei estimări mai stabile. Articolul menționează utilizarea unui Extended Kalman Filter pentru fuziunea odometriei și a IMU-ului, cu o rată de estimare de 20 Hz [25]. Această informație este importantă deoarece task-ul de manipulare depinde de poziționarea platformei. Dacă platforma nu se oprește într-o poziție repetabilă, brațul poate ajunge mai greu la obiect.

Din punct de vedere software, platforma Xplorer-A este construită în jurul ROS2 Jazzy și Nav2 [11], [12], [25]. ROS2 are rolul de middleware, adică permite comunicarea între nodurile care controlează senzorii, actuatorii, localizarea, navigarea și interfețele externe. Nav2 asigură funcțiile de navigare autonomă, cum ar fi localizarea pe hartă, planificarea traseului, controlul local și evitarea obstacolelor [12]. În articol, stack-ul include AMCL, EKF, MPPI controller și SmacPlannerHybrid, ceea ce arată că platforma este configurată pentru navigație autonomă reală, nu doar pentru deplasare manuală [25].

Tabelul 1.1.2 prezintă componentele software principale ale platformei, cu rolul lor în sistem.

Tabelul 1.1.2 - Stack software existent pe platforma Xplorer-A

Componentă software	Rol	Observații pentru integrare
ROS2 Jazzy	Middleware robotic	Baza de comunicație pentru toate modulele
Nav2	Navigare autonomă	Folosit pentru deplasarea către punctul de interes
AMCL	Localizare pe hartă	Estimează poziția robotului în mediul indoor
EKF	Fuziune odometrie și IMU	Crește stabilitatea estimării de stare
MPPI controller	Control local al mișcării	Asigură urmărirea traiectoriei planificate
SmacPlannerHybrid	Planificare globală în baseline	Generează trasee pentru navigare
ros2_control	Interfață control hardware	Gestionează controlul motoarelor și poate coexista cu brațul
TF2	Transformări între frame-uri	Esențial pentru platformă, braț și Unity

Pentru proiectul curent, cea mai importantă funcție a platformei este capacitatea de a naviga autonom către un punct de interes. Scenariul propus în lucrare presupune următoarea succesiune: platforma se deplasează cu Nav2 către o zonă în care obiectul este accesibil, se oprește, brațul execută task-ul de pick and place, iar apoi platforma poate continua navigarea. Această succesiune este

posibilă tocmai pentru că Xplorer-A are deja o arhitectură Nav2 funcțională.

Articolul introduce și o extensie semantică a navigării, bazată pe fuziunea dintre cameră și LiDAR prin metoda Angular Sector Fusion, prescurtat ASF [25]. ASF proiectează detecțiile vizuale din imagine pe sectoare unghiulare ale scanării LiDAR și estimează poziția obiectelor în cadrul hărții. Informațiile semantice sunt apoi stocate într-o hartă GeoJSON și folosite pentru a penaliza anumite muchii dintr-un graf de navigare, astfel încât Nav2 Route Server să poată alege trasee mai potrivite din punct de vedere semantic [25]. În lucrarea de față, această componentă nu este implementată ca obiectiv principal, dar demonstrează că platforma Xplorer-A este deja compatibilă cu dezvoltări complexe în ROS2.

Această observație este relevantă deoarece integrarea brațului OMX-F se face pe o platformă care a fost validată în scenarii de navigare avansată. Articolul raportează testarea pe 115 segmente de navigare, cu o rată totală de succes de 97% și 42 de evenimente de replanning în scenariile adaptive [25]. Chiar dacă aceste rezultate aparțin sistemului de rutare semantică, ele confirmă faptul că platforma are un stack de navigare matur și capabil să funcționeze pe hardware edge. Pentru lucrarea de față, acest lucru reduce riscul ca problemele de navigare de bază să blocheze integrarea brațului.

Tabelul 1.1.3 sintetizează câteva rezultate și observații din articol, interpretate prin prisma proiectului curent.

Tabelul 1.1.3 - Elemente din articol relevante pentru proiectul de integrare

Element din articol	Valoare / descriere	Relevanță pentru lucrarea curentă
Platforme testate	Xplorer-A, Xplorer-B, Xplorer-C	Confirmă existența unei familii de platforme mobile validate
Hardware edge	Raspberry Pi 5, CPU-only	Arată constrângerile reale de calcul
Senzori principali	Cameră monoculară + LiDAR 2D + IMU	Bază pentru navigare și posibile extensii de percepție
Stack software	ROS2 Jazzy + Nav2	Compatibil cu integrarea propusă
Fuziune semantică	Angular Sector Fusion	Direcție viitoare pentru localizarea obiectelor
Rutare semantică	Nav2 Route Server cu penalizări	Demonstrează extensibilitatea platformei
Evaluare experimentală	115 segmente de navigare	Platformă testată în condiții reale
Rată succes	97% total	Indică robustețea a navigării

Pentru integrarea brațului robotic, un aspect esențial este separarea clară între stack-ul existent și componentele adăugate. Stack-ul de navigare al platformei trebuie păstrat cât mai stabil, iar componentele noi trebuie adăugate la nivel de noduri ROS2 și interfețe. Această abordare este în acord cu logica articolului, unde stratul semantic este adăugat peste Nav2 fără a înlocui complet stack-ul de navigare [25]. În lucrarea de față, brațul și interfața Unity trebuie integrate în același mod: ca extensii controlate, nu ca modificări agresive ale platformei.

Tabelul 1.1.4 prezintă interfețele ROS2 ale platformei care sunt importante pentru proiectul curent. Denumirile pot fi ajustate în funcție de configurația exactă a robotului, dar rolurile sunt cele esențiale.

Tabelul 1.1.4 - Interfețe ROS2 relevante ale platformei Xplorer-A

Interfață	Tip	Rol în platformă	Utilizare în proiect
/odom	Topic	Publică odometria platformei	Vizualizare în Unity și monitorizare deplasare
/tf, /tf_static	Topicuri TF2	Transformări între frame-uri	Corelare platformă-braț-Unity
/scan	Topic	Date LiDAR 2D	Navigare și evitare obstacole
/cmd_vel	Topic	Comenzi de viteză	Control platformă, inclusiv din Unity în test
/map	Topic	Hartă de navigare	Context pentru Nav2
/amcl_pose	Topic	Poziția estimată pe hartă	Confirmarea localizării la POI
NavigateToPose	Acțiune Nav2	Trimitere obiectiv navigare	Scenariul integrat Nav2 -> MTC
/joint_states platformă/braț	Topic	Stări articulare acolo unde există	Sincronizare cu modelul 3D

În integrarea cu brațul OMX-F, frame-urile TF2 sunt foarte importante. Platforma are propriile frame-uri, precum map, odom, base_link și frame-urile senzorilor. Brațul adaugă frame-uri noi pentru baza brațului, articulații și end-effector. Pentru ca sistemul să funcționeze corect, trebuie definită transformarea statică dintre base_link al platformei și baza brațului. Dacă această transformare este greșită, MoveIt2 va planifica mișcări într-o poziție care nu corespunde montajului real, iar Unity va afișa un model nealiniat.

Un alt aspect important este influența brațului asupra navigării. Montarea brațului și a coșului poate modifica geometria robotului. Dacă robotul devine mai înalt, mai lung sau mai lat, configurația de navigare trebuie verificată pentru a se asigura că footprint-ul sau zonele de siguranță rămân corecte. În mod ideal, brațul trebuie să fie adus într-o poziție compactă înainte ca platforma să se deplaseze. Această poziție reduce riscul de coliziune și menține comportamentul robotului cât mai apropiat de cel al platformei inițiale.

Din punct de vedere al resurselor de calcul, articolul arată că rularea percepției semantice pe Raspberry Pi 5 poate duce la utilizare CPU ridicată, iar separarea inferenței pe o a doua placă poate reduce încărcarea medie de la 85% la 48% [25]. Pentru lucrarea de față, această observație este utilă deoarece bridge-ul ROS2-Unity, MoveIt2 și task-ul MTC adaugă și ele sarcini software. Chiar dacă planificarea MTC nu rulează continuu ca detecția vizuală, este necesară monitorizarea încărcării sistemului în timpul testelor, mai ales dacă Unity primește multe topicuri simultan.

În ceea ce privește relația dintre platforma Xplorer-A și interfața AR, platforma oferă datele de bază necesare pentru vizualizare: poziție, orientare, odometrie, scanare LiDAR și stare de navigare. Aceste date pot fi transmise către Unity prin ROS-TCP-Connector și pot fi reprezentate într-o scenă 3D. Dacă ulterior se dorește integrarea semantică din articol, obiectele detectate prin ASF ar putea fi afișate și ele în AR, sub formă de marcaje sau zone de atenție. Astfel, articolul nu este util doar pentru descrierea platformei, ci și pentru direcțiile viitoare ale interfeței.

Platforma Xplorer-A are un rol dublu în lucrare. Pe de o parte, este suport mecanic și software pentru brațul OMX-F. Pe de altă parte, este un exemplu de platformă educațională și de cercetare construită modular, în care pot fi adăugate funcții noi fără schimbarea completă a arhitecturii. Această modularitate este esențială pentru proiect, deoarece manipularea, bridge-ul Unity și AR sunt

dezvoltate ca extensii ale unui sistem deja funcțional.

În concluzie, Xplorer-A este o platformă potrivită pentru tema lucrării deoarece dispune de un stack ROS2/Nav2 validat, senzori suficienți pentru navigare indoor, hardware edge realist și o arhitectură modulară care permite extinderea cu manipulare robotică. Articolul analizat demonstrează capabilitățile platformei în navigare semantică și confirmă că aceasta poate susține scenarii complexe în ROS2 [25]. În lucrarea de față, această bază este folosită pentru un obiectiv diferit, și anume integrarea unui braț OMX-F și a unei interfețe de vizualizare AR pentru un scenariu de pick and place autonom.

1.2 Brațul robotic de manipulare OMX-F - prezentarea kit-ului utilizat

Brațul robotic utilizat în cadrul lucrării este OMX-F, un kit comercial dezvoltat de ROBOTIS pentru aplicații educaționale, cercetare și prototipare rapidă în robotică [19], [20], [21]. Alegerea acestui braț este justificată prin faptul că oferă o structură mecanică suficient de compactă pentru integrarea pe o platformă mobilă, dar în același timp suficient de flexibilă pentru realizarea unui task de manipulare de tip pick and place. Brațul dispune de cinci grade de libertate și de un gripper, ceea ce permite mișcări de apropiere, orientare, prindere și plasare a obiectului în zona dorită.

ROBOTIS este cunoscută în domeniul roboticii pentru actuatorii Dynamixel, pentru kit-urile educaționale și pentru suportul oferit comunității open-source [23], [29]. În cazul brațului OMX-F, avantajul principal este că producătorul pune la dispoziție nu doar componentele hardware, ci și documentație pentru asamblare, operare în ROS2, simulare și integrare cu pachetele open_manipulator [19], [20], [21], [22], [29]. Acest lucru reduce timpul necesar pentru pornirea proiectului și permite concentrarea pe integrarea cu platforma mobilă și pe scenariul aplicativ.

Kit-ul OMX-F are o structură modulară. Din punct de vedere mecanic, brațul este alcătuit din elemente de bază, segmente intermediare, suport pentru încheietură și mecanism de gripper. Din punct de vedere electronic, ansamblul este bazat pe servomotoare inteligente Dynamixel, controller OpenRB-150, cabluri de comunicație și componente de alimentare [20], [23]. Servomotoarele Dynamixel sunt importante deoarece includ feedback de poziție și pot fi controlate printr-un protocol serial dedicat, ceea ce simplifică integrarea într-un sistem ROS2.

Tabelul 1.2.1 prezintă o listă orientativă a componentelor principale ale kit-ului OMX-F. Tabelul este realizat pe baza documentației ROBOTIS și a structurii mecanice specifice brațului, fiind adaptat pentru contextul proiectului de față [20], [23], [29].

Tabelul 1.2.1 - Componente principale ale brațului OMX-F

Nr. crt.	Componentă	Descriere	Rol în ansamblu	Modalitate de procurare
1	Bază mecanică	Elementul inferior al brațului	Fixarea brațului pe suport/platformă	Kit ROBOTIS / printare 3D
2	Segment umăr	Element structural între bază și cot	Asigură ridicarea brațului	Kit ROBOTIS / printare 3D

Nr. crt.	Componentă	Descriere	Rol în ansamblu	Modalitate de procurare
3	Segment cot	Element structural intermediar	Extinde spațiul de lucru	Kit ROBOTIS / printare 3D
4	Încheietură	Modul de orientare al efectorului final	Permite poziționarea gripperului	Kit ROBOTIS / printare 3D
5	Gripper	Efector final de tip clește	Prinderea și eliberarea obiectului	Kit ROBOTIS
6	Dynamixel XL330/XL430	Actuatori inteligenți	Acționarea articulațiilor	Kit ROBOTIS
7	OpenRB-150	Controller pentru actuatori	Comunicație și control Dynamixel	Kit ROBOTIS
8	Cabluri Dynamixel	Cabluri seriale și alimentare	Interconectare actuator-controller	Kit ROBOTIS
9	Sursă / modul alimentare	Modul de alimentare dedicat	Alimentare braț și actuatori	Kit ROBOTIS
10	Elemente de fixare	Șuruburi, distanțiere, cleme	Asamblare mecanică	Kit ROBOTIS / achiziții

Din punct de vedere cinematic, brațul este potrivit pentru sarcini de laborator cu obiecte ușoare. Cele cinci grade de libertate permit poziționarea end-effectorului într-un spațiu tridimensional limitat, dar suficient pentru scenariul propus. Gripperul permite prinderea unor obiecte de dimensiuni reduse, cum ar fi cuburi printate 3D, role mici, obiecte cilindrice sau piese ușoare. În lucrarea de față, scenariul este orientat către preluarea unui obiect de pe sol și plasarea lui într-un coș montat pe platformă.

Este important de menționat că acest braț nu este proiectat pentru sarcini industriale grele. Sarcina utilă, rigiditatea mecanică și precizia sunt adaptate pentru aplicații educaționale și de cercetare. Totuși, pentru proiectul propus, aceste limitări nu reprezintă un dezavantaj major, deoarece scopul nu este manipularea unor obiecte grele, ci demonstrarea unui flux complet de integrare: planificare, execuție, vizualizare și coordonare cu navigarea autonomă.

Tabelul 1.2.2 sintetizează caracteristicile relevante ale brațului pentru proiectarea sistemului. Valorile exacte pot fi actualizate în lucrarea finală în funcție de configurația fizică măsurată și de documentația completă a kit-ului utilizat.

Tabelul 1.2.2 - Caracteristici tehnice relevante pentru integrarea OMX-F

Caracteristică	Valoare / observație	Impact asupra proiectului
Număr grade de libertate	5 DOF + gripper	Permite task-uri simple de pick and place
Tip actuatori	Dynamixel seria X	Control cu feedback și integrare ROS2
Efector final	Gripper mecanic	Prindere obiecte ușoare
Structură mecanică	Modulară, compactă	Poate fi montată pe platformă mobilă
Control software	Pachete open_manipulator ROS2	Integrare cu ros2_control și MoveIt2
Simulare	Gazebo / model URDF	Testare înainte de hardware fizic
Alimentare	Modul dedicat pentru actuatori	Necesită analiză electrică pe platformă
Limitare principală	Workspace și sarcină utilă reduse	Obiectele și coșul trebuie poziționate corect

Integrarea brațului pe platforma mobilă impune o analiză suplimentară față de utilizarea lui pe banc. Pe banc, baza brațului este fixă, iar obiectele pot fi poziționate în zona optimă de lucru. Pe platforma mobilă, baza brațului se deplasează odată cu robotul, iar spațiul de lucru este influențat de poziția de montaj, înălțimea platformei, poziția coșului și posibilitatea de a ajunge la obiectul

aflat pe sol. De aceea, în acest proiect, montarea brațului nu este tratată ca o simplă fixare mecanică, ci ca o decizie de proiectare care influențează întregul sistem.

Din punct de vedere software, kit-ul OMX-F oferă o bază de lucru prin pachetele `open_manipulator` și prin modelele disponibile în repository-urile ROBOTIS [29]. Aceste pachete includ descrieri ale robotului, fișiere de configurare, launch files, exemple de operare și integrare cu instrumente din ecosistemul ROS2. Pentru proiectul de față, aceste resurse sunt folosite ca punct de plecare, însă trebuie adaptate pentru configurația în care brațul este montat pe o platformă mobilă.

Un element central în această integrare este descrierea URDF a robotului. URDF-ul definește structura robotului prin link-uri, articulații, transformări, limite și proprietăți vizuale sau de coliziune [31]. În cazul brațului montat pe platformă, modelul trebuie să includă corect poziția bazei brațului față de cadrul platformei. Dacă această poziție este greșită, planificarea mișcării poate produce traiectorii care par corecte în simulator, dar nu corespund sistemului real. Din acest motiv, etapa de proiectare trebuie să includă verificarea transformărilor și a scenei de coliziune.

Pentru controlul hardware, `ros2_control` are rolul de a crea o interfață standard între nodurile ROS2 și actuatorii robotului [30]. Prin intermediul controllerelor, traiectoriile calculate de MoveIt2 pot fi transmise către actuatori sub formă de comenzi. În proiect, acest lucru este important deoarece se dorește executarea unui task planificat, nu doar mișcarea manuală a fiecărei articulații.

Interfețele ROS2 relevante pentru braț sunt prezentate în Tabelul 1.2.3. Acestea sunt importante deoarece reprezintă punctele prin care brațul comunică cu restul sistemului.

Tabelul 1.2.3 - Interfețe ROS2 relevante pentru brațul OMX-F

Tip interfață	Denumire orientativă	Descriere	Rol în sistem
Nod	<code>robot_state_publisher</code>	Publică transformările robotului pe baza URDF și a stării articulațiilor	Vizualizare și planificare
Nod	<code>controller_manager</code>	Gestionează controllerele <code>ros2_control</code>	Pornire și monitorizare controlere
Nod	<code>joint_trajectory_executor</code>	Execută traiectorii pe braț	Trimitere comenzi către actuatori
Topic	<code>/joint_states</code>	Publică poziția, viteza și efortul articulațiilor	Feedback pentru MoveIt2 și Unity
Topic	<code>/tf</code> și <code>/tf_static</code>	Publică transformări între frame-uri	Coerență geometrică în sistem
Acțiune	<code>/arm_controller/follow_joint_trajectory</code>	Primește traiectorii pentru braț	Execuție mișcări planificate
Serviciu	<code>/controller_manager/list_controllers</code>	Listează controllerele disponibile	Diagnosticare
Serviciu	<code>/controller_manager/switch_controller</code>	Activează/dezactivează controllere	Configurare și testare

Pentru task-ul de pick and place, informațiile din `/joint_states` sunt necesare atât pentru MoveIt2, cât și pentru vizualizarea în Unity. Dacă poziția articulațiilor este transmisă corect, scena Unity poate reproduce mișcarea brațului, iar utilizatorul poate urmări starea sistemului într-un mod vizual. În același timp,

Movelt2 folosește starea curentă a brațului pentru a calcula traiectorii realizabile [13].

În concluzie, brațul OMX-F este potrivit pentru lucrarea de față deoarece oferă un echilibru între accesibilitate, suport software și posibilitatea de integrare pe o platformă mobilă. Contribuția lucrării nu constă în proiectarea brațului, ci în utilizarea sa într-o configurație nouă, adaptată unui sistem mobil autonom cu interfață AR.

Pentru a înțelege mai bine rolul brațului în sistemul integrat, este utilă și o analiză a limitărilor sale. În primul rând, lungimea segmentelor și cuplul actuatorilor impun o limită asupra obiectelor care pot fi manipulate. Într-un proiect de laborator, această limitare este acceptabilă, deoarece obiectele pot fi alese astfel încât să se încadreze în capabilitățile brațului. În schimb, într-o aplicație industrială, ar fi necesară o analiză mai detaliată a sarcinii utile, a momentelor în articulații și a rigidității structurii [50], [51].

În al doilea rând, precizia task-ului depinde de mai mulți factori: calibrarea mecanică, jocurile din articulații, calitatea modelului URDF, rezoluția controlului și poziția reală a obiectului. Dacă obiectul este definit în scenă într-o poziție ideală, dar în realitate este deplasat cu câțiva centimetri, task-ul poate eșua. Din acest motiv, în etapa de proiectare trebuie aleasă o clasă de obiecte care permite o toleranță rezonabilă la prindere. Obiectele prea mici sau prea alunecoase ar face dificilă validarea sistemului, chiar dacă arhitectura software este corectă.

În al treilea rând, gripperul are o geometrie simplă și nu poate adapta forma de prindere la orice obiect. Pentru această lucrare, obiectele recomandate sunt obiecte ușoare, cu forme regulate și dimensiuni compatibile cu deschiderea gripperului. Exemple potrivite sunt cuburi printate 3D, cilindri mici, role ușoare sau piese cu suprafețe laterale paralele. Alegerea obiectului trebuie să susțină obiectivul principal al lucrării, adică integrarea sistemului, nu testarea limitelor mecanice ale gripperului.

Tabelul 1.2.4 - Criterii de alegere a obiectelor pentru task-ul pick and place

Criteriu	Recomandare	Motiv
Masă	Obiect ușor	Reduce solicitarea actuatorilor
Formă	Cub, cilindru sau prismă regulată	Ușurează definirea poziției de grasp
Dimensiune	Compatibilă cu deschiderea gripperului	Permite prindere stabilă
Suprafață	Nu foarte alunecoasă	Crește rata de succes
Culoare	Contrast față de mediu	Ajută la documentare și posibilă percepție
Poziționare	Repetabilă în testele inițiale	Permite validarea sistematică

Analiza obiectelor este importantă deoarece influențează direct parametrii MTC. Un obiect cilindric poate necesita o orientare diferită a gripperului față de un cub. Un obiect mai înalt poate necesita o poziție de pre-grasp mai ridicată. Un obiect mai lat poate necesita deschiderea completă a gripperului. Prin urmare, înainte de implementare, obiectul de test trebuie definit ca parte a sistemului, nu ca detaliu secundar.

Un alt element important este poziția camerei, dacă se folosește o cameră pentru vizualizare sau pentru dezvoltări ulterioare de percepție. În configurația minimă, poziția obiectului poate fi definită manual, iar camera poate avea doar rol

de observare. Într-o configurație extinsă, camera poate furniza informații despre poziția obiectului, iar atunci calibrarea dintre camera, braț și platformă devine esențială. Problemele de viziune robotică și control vizual sunt tratate pe larg în literatura de specialitate, însă în această lucrare ele sunt păstrate ca direcție ulterioară, pentru a nu muta accentul de pe integrarea sistemului [52]. Pentru lucrarea de față, proiectarea lasă deschisă această extensie, dar nu condiționează funcționarea task-ului de o detecție vizuală completă.

1.3 Cadrul MoveIt2 și MoveIt2 Task Constructor

MoveIt2 reprezintă framework-ul principal utilizat pentru planificarea mișcării brațului robotic în ecosistemul ROS2 [13]. Rolul său este de a permite definirea modelului cinematic, calculul traiectoriilor, verificarea coliziunilor și interfațarea cu controllerele de execuție. Într-un sistem robotic simplu, comanda articulațiilor poate fi realizată manual sau prin poziții predefinite. Într-un task de pick and place, această abordare devine limitată, deoarece brațul trebuie să ajungă în poziții diferite, să evite coliziuni și să respecte constrângeri geometrice.

MoveIt2 pornește de la modelul robotului definit prin URDF și SRDF. URDF-ul descrie structura fizică a robotului, iar SRDF-ul completează informația necesară pentru planificare, cum ar fi grupurile de articulații, end-effectorul, pozițiile predefinite și coliziunile permise [31], [32]. Pentru brațul OMX-F, grupul de planificare principal este grupul brațului, iar gripperul este tratat ca efector final. Această separare este utilă deoarece mișcarea brațului și acțiunea gripperului au roluri diferite în task.

Planificarea mișcării presupune găsirea unei traiectorii valide între starea curentă și o stare țintă. Traectoria trebuie să respecte limitele articulațiilor, să fie compatibilă cu geometria robotului și să evite coliziunile. MoveIt2 poate utiliza planificatoare precum OMPL pentru găsirea traiectoriilor în spațiul articulațiilor sau în spațiul cartezian [40]. În contextul lucrării, această capacitate este importantă deoarece brațul trebuie să se apropie de obiect, să îl prindă și să îl ducă în coș fără să lovească platforma sau coșul.

MoveIt2 Task Constructor, prescurtat MTC, este o extensie a MoveIt2 care permite proiectarea task-urilor complexe sub forma unor etape sau stagii [14]. În loc să se definească o singură mișcare de la punctul A la punctul B, task-ul este împărțit în pași logici. Fiecare pas are un rol clar, iar rezultatul unui pas devine intrare pentru pasul următor. Această abordare este foarte potrivită pentru pick and place, deoarece task-ul este în mod natural secvențial.

Tabelul 1.3.1 prezintă diferența dintre utilizarea MoveIt2 ca framework general de planificare și utilizarea MoveIt2 Task Constructor pentru task-uri structurate.

Tabelul 1.3.1 - Comparație între MoveIt2 și MoveIt2 Task Constructor

Criteriu	MoveIt2	MoveIt2 Task Constructor
Rol principal	Planificare mișcare între stări	Planificare task-uri multi-stagiu
Nivel de abstractizare	Traectorii individuale	Secvențe logice de acțiuni
Utilizare tipică	Move to pose, planificare articulații	Pick and place, asamblare, task-uri structurate
Diagnosticare	Eșec la nivel de planificare globală	Eșec identificabil pe stagii
Avantaj principal	Flexibilitate și integrare ROS2	Organizare clară a task-ului

Criteriu	Movelt2	Movelt2 Task Constructor
Relevanță în proiect	Calcul traiectorii pentru OMX-F	Definirea completă a scenariului pick and place

Pentru scenariul acestei lucrări, task-ul este definit astfel: obiectul se află pe sol, în apropierea platformei; brațul se deplasează către o poziție de pre-grasp; gripperul se deschide; brațul coboară sau se apropie de obiect; gripperul se închide; obiectul este ridicat; brațul se deplasează către coș; gripperul se deschide; obiectul este eliberat; brațul revine într-o poziție sigură. Această succesiune poate fi descrisă natural prin MTC.

Tabelul 1.3.2 prezintă stagiile principale proiectate pentru task-ul de pick and place.

Tabelul 1.3.2 - Stagii propuse pentru task-ul pick and place cu MTC

Nr.	Stagiu MTC	Rol	Observații de proiectare
1	Current State	Preia starea curentă a brațului	Necesită /joint_states valid
2	Open Gripper	Deschide gripperul înainte de prindere	Poziție sigură pentru apropiere
3	Move To Pre-Grasp	Mută brațul lângă obiect	Poziția trebuie să evite coliziunile
4	Approach Object	Apropie end-effectorul de obiect	Mișcare carteziană controlată
5	Grasp	Închide gripperul pe obiect	Depinde de dimensiunea obiectului
6	Lift	Ridică obiectul	Evită contactul cu solul
7	Move To Place	Mută obiectul deasupra coșului	Traietorie fără coliziuni cu platforma
8	Place	Coboară și poziționează obiectul în coș	Depinde de poziția coșului
9	Open Gripper	Eliberează obiectul	Confirmă finalizarea task-ului
10	Return Home	Revine într-o poziție de siguranță	Pregătește reluarea navigării

Un avantaj important al MTC este faptul că fiecare stagiul poate fi testat și ajustat separat. Dacă robotul nu reușește să prindă obiectul, se poate analiza dacă problema apare la poziția de pre-grasp, la orientarea gripperului, la adâncimea de apropiere sau la închiderea gripperului. Dacă robotul nu reușește să plaseze obiectul în coș, problema poate fi legată de poziția coșului, de înălțimea de place sau de coliziuni cu marginile coșului. Această separare face procesul de dezvoltare mai clar.

În proiectarea task-ului trebuie stabilite mai multe elemente geometrice. Primul este poziția obiectului față de baza platformei sau față de un frame de referință. Al doilea este orientarea gripperului în momentul prinderii. Al treilea este poziția coșului față de baza brațului. Al patrulea este înălțimea minimă de ridicare pentru a evita contactul cu solul și cu platforma. Toate aceste valori trebuie să fie compatibile cu spațiul de lucru al brațului.

Un alt aspect important este scena de coliziune. Pentru ca Movelt2 să poată evita obstacolele, trebuie definite obiectele relevante: platforma mobilă, suportul brațului, coșul, obiectul de manipulat și eventualele limite ale mediului [13]. Dacă aceste obiecte nu sunt introduse corect, planificatorul poate calcula traiectorii care, în realitate, ar produce coliziuni. În schimb, dacă scena de coliziune este prea restrictivă, planificatorul poate eșua chiar și atunci când mișcarea ar fi

posibilă.

În cadrul lucrării, proiectarea task-ului MTC este realizată înainte de implementarea finală pe robotul fizic. Această ordine este necesară deoarece permite definirea parametrilor principali și verificarea lor în simulare. Gazebo poate fi folosit pentru a testa modelul și pentru a observa dacă brațul poate executa succesiunea de mișcări fără riscuri [18], [44]. Ulterior, parametrii trebuie ajustați pe hardware fizic.

Movelt2 și MTC sunt alese în locul unei abordări bazate doar pe comenzi directe deoarece task-ul propus are nevoie de planificare, nu doar de execuție rigidă. Dacă poziția obiectului sau a coșului se modifică ușor, o secvență fixă de valori ale articulațiilor poate deveni nepotrivită. Prin planificare, sistemul poate calcula traiectorii în funcție de configurația curentă. Aceasta nu înseamnă că sistemul devine complet robust la orice variație, dar oferă o flexibilitate mai mare decât o comandă hardcodată.

În concluzie, Movelt2 oferă baza pentru planificarea mișcării, iar Movelt2 Task Constructor oferă structura logică pentru task-ul de pick and place. Împreună, aceste două componente permit proiectarea unui scenariu de manipulare clar, testabil și integrabil cu platforma mobilă și cu interfața Unity.

Pentru proiectarea mai detaliată a task-ului, fiecare stagi MTC trebuie asociat cu o condiție de intrare, o acțiune și o condiție de ieșire. De exemplu, stagiul Open Gripper are ca intrare starea curentă a robotului și ca ieșire confirmarea că gripperul se află într-o poziție deschisă. Stagiul Move To Pre-Grasp are ca intrare poziția obiectului și configurația curentă a brațului, iar ca ieșire o poziție validă înainte de prindere. Această gândire pe intrări și ieșiri ajută la depanare, deoarece fiecare etapă poate fi verificată independent.

Tabelul 1.3.3 - Condiții de intrare și ieșire pentru stagiile MTC

Stagiu	Condiție de intrare	Acțiune	Condiție de ieșire
Current State	Braț conectat, /joint_states activ	Citește starea curentă	Stare validă disponibilă
Open Gripper	Controller gripper activ	Deschide efectorul final	Gripper deschis
Move To Pre-Grasp	Obiect definit în scenă	Planifică apropierea	Poziție pre-grasp atinsă
Approach Object	Pre-grasp valid	Mișcare carteziană către obiect	End-effector în zona de prindere
Grasp	Gripper aliniat	Închide gripperul	Obiect prins
Lift	Obiect prins	Ridicare verticală	Obiect deasupra solului
Move To Place	Coș definit în scenă	Planifică deplasarea	Poziție de place atinsă
Place	Poziție de place validă	Coboară și eliberează	Obiect eliberat
Return Home	Task finalizat	Planifică revenirea	Poziție sigură atinsă

În proiectarea parametrilor MTC trebuie evitată alegerea unor valori prea optimiste. De exemplu, o distanță de apropiere foarte mică poate părea eficientă, dar poate reduce toleranța la erori de poziționare. O înălțime de ridicare prea mică poate produce coliziuni cu solul sau cu marginea coșului. O orientare a gripperului prea strictă poate reduce numărul de soluții de cinematică inversă. Din acest motiv, parametrii inițiali trebuie aleși conservator și ajustați pe baza testelor.

Un alt aspect important este definirea obiectelor de coliziune. Într-un task de pick and place, obiectul este inițial un obstacol în scenă, apoi devine atașat de gripper, iar după eliberare devine din nou obiect independent. MTC permite modelarea acestui flux, dar trebuie configurat corect. Dacă obiectul nu este atașat în momentul ridicării, planificatorul poate considera că acesta a rămas pe sol. Dacă obiectul rămâne atașat după place, scenele următoare pot fi greșite. Acest tip de detaliu arată de ce MTC este potrivit pentru task-uri structurate, dar necesită o proiectare atentă [14].

În ceea ce privește algoritmi de planificare, MoveIt2 poate utiliza planificatori bazați pe sampling, cum sunt cei din OMPL [40]. Acești algoritmi sunt potriviți pentru spații de configurație complexe, dar rezultatul poate varia în funcție de parametri și de timpul de planificare [53]. Pentru un braț mic, timpii de planificare pot fi menținuți rezonabili, însă scenele cu multe coliziuni sau constrângeri stricte pot crește durata de calcul. În lucrare, timpul de planificare trebuie tratat ca o metrică de testare, nu doar ca un detaliu intern.

Pentru validarea task-ului, se recomandă o abordare în trepte. Prima treaptă este planificarea fără obiect, pentru a verifica dacă brațul se mișcă în spațiul util. A doua treaptă este planificarea cu obiect și coș în scenă, dar fără execuție hardware. A treia treaptă este execuția în simulare. A patra treaptă este execuția pe brațul fizic montat pe banc. Ultima treaptă este execuția pe brațul montat pe platformă. Această progresie reduce riscul și face mai ușoară identificarea cauzelor de eșec.

1.4 Cadrul ROS2-Unity și Realitatea Augmentată

Componenta ROS2-Unity are rolul de a conecta sistemul robotic cu o interfață grafică dezvoltată în Unity. Într-un sistem robotic clasic, dezvoltatorul urmărește starea robotului prin terminale, RViz, loguri și instrumente ROS2. Aceste instrumente sunt foarte utile pentru depanare, dar nu sunt întotdeauna potrivite pentru un operator care dorește să înțeleagă rapid ce face robotul. De aceea, în lucrarea de față se propune o scenă Unity care să reprezinte robotul, brațul, coșul și obiectul manipulat.

Unity este o platformă de dezvoltare 3D folosită frecvent pentru aplicații interactive, simulări și aplicații XR [16]. În contextul roboticii, Unity poate fi folosită ca interfață de vizualizare, ca mediu de simulare grafică sau ca bază pentru aplicații AR. Pentru comunicarea cu ROS2 se pot utiliza pachete precum Unity Robotics Hub și ROS-TCP-Connector [9], [15]. În arhitectura propusă, ROS2 rămâne partea de control și comunicare robotică, iar Unity primește informații și le reprezintă vizual.

ROS-TCP-Connector funcționează împreună cu un server ROS-TCP-Endpoint care rulează în ecosistemul ROS și permite schimbul de mesaje între ROS2 și Unity [15], [34]. Această arhitectură este utilă deoarece evită implementarea manuală a protocolului ROS în Unity. Mesajele ROS sunt convertite într-o formă accesibilă aplicației Unity, iar aplicația poate publica sau subscrie la topicuri relevante.

Pentru proiectul de față, bridge-ul ROS2-Unity este proiectat pentru două direcții de comunicație. Prima direcție este ROS2 către Unity, folosită pentru vizualizarea stării robotului. A doua direcție este Unity către ROS2, folosită pentru trimiterea unor comenzi de control sau pentru declanșarea unor acțiuni simple. În

faza inițială, accentul cade pe vizualizare și pe demonstrarea controlului de bază, nu pe controlul complet al task-ului de manipulare din AR.

Tabelul 1.4.1 prezintă topicurile și informațiile care pot fi transmise între ROS2 și Unity.

Tabelul 1.4.1 - Fluxuri de date propuse pentru ROS2-Unity bridge

Direcție	Topic / mesaj	Date transmise	Utilizare în Unity
ROS2 -> Unity	/joint_states	Poziții articulații braț	Animarea modelului OMX-F
ROS2 -> Unity	/tf, /tf_static	Transformări între frame-uri	Poziționarea elementelor 3D
ROS2 -> Unity	/odom	Poziția platformei mobile	Mișcarea robotului în scenă
ROS2 -> Unity	/scan	Date LiDAR	Vizualizare obstacole / mediu
ROS2 -> Unity	/camera/image_raw	Imagine cameră	Panou video sau textură
ROS2 -> Unity	/mtc/status	Starea task-ului	Feedback pentru operator
Unity -> ROS2	/cmd_vel	Comenzi de viteză	Control platformă în demonstrații
Unity -> ROS2	/unity/trigger_task	Comandă de pornire task	Declanșare scenariu integrat
Unity -> ROS2	/unity/ar_goal	Poziții selectate în AR	Direcție viitoare pentru control AR

Realitatea augmentată extinde această interfață prin suprapunerea informațiilor digitale peste mediul real. În cazul Meta Quest, aplicația Unity poate fi rulată într-un mod de realitate mixtă, unde utilizatorul vede mediul real și elementele virtuale simultan [17], [37]. Pentru robotica mobilă, această posibilitate este importantă deoarece operatorul poate vedea robotul fizic și informațiile despre acesta în același spațiu.

În proiectarea interfeței AR trebuie tratate trei probleme principale. Prima este calibrarea spațiului real-virtual. Modelul din Unity trebuie să fie aliniat cu robotul fizic sau cel puțin să fie reprezentat într-o poziție coerentă față de utilizator. A doua este latența. Dacă datele ajung cu întârziere în Unity, modelul virtual poate rămâne în urma robotului real. A treia este claritatea interfeței. O interfață AR prea încărcată poate deveni greu de utilizat, mai ales într-un mediu în care operatorul trebuie să fie atent și la robotul fizic.

Pentru a evita această problemă, interfața este proiectată pe niveluri. Nivelul de bază afișează starea generală: robot conectat, navigare activă, task MTC activ, eroare sau finalizare. Nivelul tehnic afișează informații suplimentare: poziții articulații, latență bridge, starea topicurilor și durata task-ului. Nivelul de control permite trimiterea unor comenzi simple sau declanșarea scenariului. Această separare permite adaptarea interfeței la tipul de utilizator.

Tabelul 1.4.2 prezintă elementele principale proiectate pentru interfața AR.

Tabelul 1.4.2 - Elemente propuse pentru interfața AR

Element interfață	Rol	Utilizator vizat	Observații
Model 3D robot	Reprezentarea platformei și a brațului	Operator / dezvoltator	Sincronizat cu ROS2
Panou stare sistem	Afișează starea generală	Operator	Simplu și vizibil
Panou task MTC	Afișează stagiul curent	Dezvoltator / operator	Util pentru diagnosticare
Indicator obiect	Marchează obiectul de manipulat	Operator	Poate fi extins cu percepție
Indicator coș	Marchează zona de place	Operator	Ajută la înțelegerea task-ului
Panou control	Comenzi de bază	Operator autorizat	Control limitat pentru siguranță

Element interfață	Rol	Utilizator vizat	Observații
Vizualizare traseu	Traectorie platformă / braț	Dezvoltator	Utilă în testare
Feedback eroare	Mesaje vizuale clare	Operator	Reduce interpretarea greșită

În cadrul lucrării, componenta AR este proiectată ca o extensie a scenei Unity. Aceasta înseamnă că baza de vizualizare este comună: modelul robotului, topicurile ROS2 și logica de actualizare sunt folosite atât în Unity desktop, cât și în aplicația AR. Diferența este modul de afișare și interacțiune. Pe desktop, utilizatorul urmărește scena pe monitor. În AR, utilizatorul vede scena în spațiul real prin Meta Quest.

O direcție importantă, menționată și în introducere, este colectarea demonstrațiilor prin AR. În această direcție, mișcările mâinii sau ale controlerului din spațiul AR ar putea fi transformate în demonstrații pentru braț. Această abordare nu este implementată complet în această lucrare, dar arhitectura propusă permite o extensie ulterioară. Pentru aceasta ar fi necesară capturarea poziției controlerului, transformarea în frame-ul robotului, filtrarea mișcărilor, validarea cinematică și salvarea datelor într-un format compatibil cu un pipeline de învățare.

În concluzie, ROS2-Unity bridge-ul este proiectat ca o punte între sistemul robotic și utilizator. El nu înlocuiește instrumentele ROS2, ci le completează printr-o reprezentare vizuală mai accesibilă. Componenta AR adaugă o direcție modernă de interacțiune și creează baza pentru extinderea sistemului către control intuitiv și colectare de demonstrații.

În proiectarea interfeței AR este util să fie respectate câteva principii de bază ale interacțiunii XR. În primul rând, elementele virtuale trebuie să fie plasate astfel încât să nu acopere zonele critice ale mediului real. Dacă panourile sunt prea mari sau sunt poziționate în fața robotului, utilizatorul poate pierde din vedere mișcările fizice. În al doilea rând, informațiile trebuie grupate logic. Starea generală a sistemului trebuie să fie vizibilă rapid, iar detaliile tehnice trebuie accesate doar când sunt necesare. În al treilea rând, inputul utilizatorului trebuie limitat pentru a preveni comenzi accidentale.

OpenXR și XR Interaction Toolkit oferă concepte și instrumente utile pentru dezvoltarea aplicațiilor XR în Unity [35], [36]. Chiar dacă implementarea completă a controlului prin AR nu este obiectivul principal al lucrării, proiectarea trebuie să țină cont de aceste standarde, deoarece ele permit portabilitate și o arhitectură mai ușor de extins. În cazul Meta Quest, Meta XR Core SDK oferă componente specifice pentru realitate mixtă și interacțiune cu dispozitivul [37].

Tabelul 1.4.3 - Principii de proiectare pentru interfața AR

Principiu	Aplicare în proiect	Beneficiu
Vizibilitate clară	Panou de stare simplu, culori distincte	Operatorul înțelege rapid sistemul
Încărcare vizuală redusă	Detaliile tehnice sunt afișate doar la cerere	Reduce oboseala și confuzia
Comenzi confirmate	Acțiunile importante necesită confirmare	Reduce riscul de declanșare accidentală
Separare control/monitorizare	AR monitorizează inițial, nu controlează complet	Crește siguranța
Feedback imediat	Starea task-ului se schimbă vizual	Utilizatorul vede progresul
Coerență spațială	Modelul virtual respectă poziția reală	Crește încrederea în interfață

Într-o etapă viitoare, interfața AR poate deveni și un instrument de definire a task-urilor. Utilizatorul ar putea indica în spațiu poziția obiectului sau poziția de place, iar aceste informații ar putea fi trimise către ROS2. Pentru ca acest lucru să funcționeze, este necesară o transformare corectă între coordonatele AR și frame-urile robotului. Această problemă este similară cu calibrarea dintre o cameră și un robot, dar devine mai complexă deoarece utilizatorul se poate deplasa în spațiul real.

În cadrul acestei lucrări, abordarea aleasă este prudentă: AR este introdusă mai întâi ca mediu de vizualizare. Această decizie este justificată de faptul că vizualizarea are risc redus și permite validarea pipeline-ului ROS2-Unity-Meta Quest. După ce acest pipeline este stabil, se pot adăuga funcții de control sau colectare de demonstrații. Astfel, proiectarea evită supradimensionarea obiectivului și păstrează o direcție clară de dezvoltare.

1.5 Cerințe și restricții de proiectare

Proiectarea sistemului integrat trebuie să pornească de la cerințele funcționale și de la restricțiile impuse de componentele disponibile. Într-un proiect robotic, cerințele definesc ce trebuie să facă sistemul, iar restricțiile definesc limitele în care sistemul poate fi realizat. Dacă aceste elemente nu sunt clar stabilite, implementarea poate ajunge să urmărească obiective neclare sau imposibil de validat.

Cerința funcțională principală este realizarea unui scenariu de pick and place autonom. Robotul mobil trebuie să se deplaseze către o zonă de interes, să se poziționeze într-o configurație potrivită, iar brațul OMX-F trebuie să preia un obiect de pe sol și să îl plaseze într-un coș montat pe platformă. După finalizarea task-ului, platforma trebuie să poată continua navigarea. Acest flux este important deoarece demonstrează integrarea dintre navigare și manipulare.

A doua cerință funcțională este vizualizarea stării sistemului în Unity și, parțial, în realitate augmentată. Utilizatorul trebuie să poată observa poziția platformei, poziția brațului, starea task-ului și eventualele erori. Această vizualizare trebuie să fie suficient de clară pentru a fi utilă în timpul testării și suficient de modulară pentru a permite extinderea către AR.

A treia cerință este păstrarea compatibilității cu ecosistemul ROS2 Jazzy existent. Platforma mobilă funcționează deja cu ROS2 și Nav2, iar lucrarea nu urmărește modificarea fundamentală a stack-ului de navigare [11], [12]. Integrarea brațului și a interfeței Unity trebuie realizată prin noduri, topicuri, acțiuni și servicii compatibile cu această arhitectură.

Tabelul 1.5.1 sintetizează cerințele funcționale ale sistemului.

Tabelul 1.5.1 - Cerințe funcționale ale sistemului integrat

Cod	Cerință funcțională	Descriere	Criteriu de validare
CF1	Navigare către punct de interes	Platforma ajunge într-o zonă definită	Goal Nav2 finalizat cu succes
CF2	Poziționare pentru manipulare	Robotul se oprește într-o poziție accesibilă brațului	Obiectul se află în workspace
CF3	Pick and place	Brațul ridică obiectul și îl plasează în coș	Obiectul ajunge în coș

Cod	Cerință funcțională	Descriere	Criteriu de validare
CF4	Reîntoarcere / reluare navigare	Platforma continuă scenariul după task	Nav2 primește următorul goal
CF5	Vizualizare Unity	Starea robotului este reprezentată în scenă	Model sincronizat cu ROS2
CF6	Demonstrație AR parțială	Modelul este vizibil în realitate mixtă	Aplicație rulată pe Meta Quest
CF7	Raportare stare task	Sistemul afișează stagiul curent	Feedback disponibil în Unity

Restricțiile hardware sunt legate în primul rând de spațiul de lucru al brațului. OMX-F are o rază de acțiune limitată, iar obiectul trebuie să se afle într-o zonă accesibilă. În același timp, coșul trebuie montat astfel încât brațul să poată plasa obiectul în interior fără coliziuni cu marginile. Poziția coșului nu trebuie să afecteze deplasarea platformei sau accesul la alte componente.

O altă restricție hardware este stabilitatea platformei. Montarea brațului adaugă masă și poate muta centrul de greutate. Dacă brațul este montat lateral sau prea sus, platforma poate deveni mai instabilă în timpul deplasării sau în timpul mișcărilor brațului. De aceea, alegerea amplasamentului trebuie să ia în calcul atât workspace-ul, cât și stabilitatea.

Restricțiile software sunt legate de compatibilitatea dintre ROS2 Jazzy, MoveIt2, pachetele open_manipulator, Unity și ROS-TCP-Connector [11], [13], [15], [29]. Versiunile pachetelor trebuie să fie compatibile, iar mesajele transmise între ROS2 și Unity trebuie să fie generate corect. În plus, bridge-ul trebuie să poată transmite date cu o frecvență suficientă pentru o vizualizare fluidă.

Restricțiile de timp sunt importante mai ales pentru vizualizare și control. Pentru task-ul MTC, planificarea poate dura câteva secunde, ceea ce este acceptabil într-un scenariu de laborator. Pentru vizualizarea Unity, întârzierea trebuie să fie mai mică, astfel încât utilizatorul să perceapă mișcarea ca fiind sincronizată. Pentru AR, latența trebuie tratată cu și mai multă atenție, deoarece decalajul dintre modelul virtual și robotul fizic poate afecta încrederea utilizatorului.

Tabelul 1.5.2 prezintă restricțiile principale de proiectare.

Tabelul 1.5.2 - Restricții de proiectare

Domeniu	Restricție	Efect asupra proiectării	Măsură propusă
Hardware	Workspace limitat al brațului	Obiectul și coșul trebuie poziționate precis	Studiu de amplasament
Hardware	Centru de greutate modificat	Poate afecta stabilitatea platformei	Montaj cât mai echilibrat
Hardware	Alimentare actuatori	Consum suplimentar pe platformă	Sursă dedicată / analiză electrică
Software	Compatibilitate ROS2 Jazzy	Pachetele trebuie să ruleze în aceeași distribuție	Testare incrementală
Software	Integrare MoveIt2-MTC	Necesită URDF/SRDF corect	Validare în RViz și Gazebo
Comunicație	Latență ROS2-Unity	Vizualizare decalată	Limitarea topicurilor și frecvențelor
AR	Calibrare real-virtual	Model virtual nealiniat	Procedură de calibrare
Siguranță	Mișcări robot în mediu real	Risc de coliziune	Scene de coliziune și poziții sigure

O cerință importantă este ca sistemul să fie dezvoltat incremental. Nu este realist ca toate componentele să fie integrate direct în forma finală. O abordare sigură este următoarea: verificarea brațului separat, verificarea platformei separat, testarea task-ului în simulare, testarea bridge-ului Unity cu date simple, integrarea brațului pe platformă, testarea MTC pe robot static și abia apoi integrarea cu navigarea Nav2. Această ordine reduce riscurile și face mai ușoară identificarea problemelor.

În concluzie, cerințele și restricțiile definesc limitele proiectului. Ele arată ce trebuie demonstrat și ce compromisuri trebuie acceptate. Pentru lucrarea de față, cele mai importante constrângeri sunt workspace-ul brațului, poziția coșului, compatibilitatea ROS2 și latența bridge-ului Unity/AR.

Pe lângă cerințele funcționale, sistemul trebuie să respecte și cerințe nefuncționale. Acestea nu descriu direct ce acțiune execută robotul, ci calitatea execuției. Într-un proiect robotic, cerințele nefuncționale sunt importante deoarece un sistem poate executa task-ul o singură dată, dar să nu fie suficient de repetabil sau sigur pentru a fi considerat valid.

Tabelul 1.5.3 - Cerințe nefuncționale și metrici de evaluare

Cod	Cerință nefuncțională	Metrică propusă	Observații
CNF1	Repetabilitate pick and place	Rată de succes (%)	Măsurată pe mai multe rulări
CNF2	Timp de execuție acceptabil	Durată ciclu (s)	Include planificare și execuție
CNF3	Precizie poziționare	Eroare end-effector (mm)	Estimată prin măsurători în testare
CNF4	Stabilitate comunicație	Pierderi mesaje / deconectări	Relevant pentru Unity bridge
CNF5	Latență vizualizare	Întârziere ROS2-Unity (ms)	Importantă pentru AR
CNF6	Siguranță	Număr coliziuni / opriri	Trebuie menținut la zero în testare
CNF7	Modularitate	Posibilitatea testării separate	Validată prin teste pe subsisteme

Siguranța trebuie analizată chiar dacă sistemul este un prototip de laborator. Standardele pentru roboți industriali și colaborativi, cum ar fi ISO 10218 și ISO/TS 15066, subliniază importanța limitării riscurilor atunci când roboții se află în apropierea oamenilor [46], [47]. Lucrarea de față nu certifică sistemul conform acestor standarde, dar principiile lor sunt relevante: mișcările trebuie să fie previzibile, operatorul trebuie să poată opri sistemul, iar zonele de coliziune trebuie tratate cu atenție.

Din acest motiv, în etapa de proiectare se recomandă includerea unor măsuri simple de siguranță: viteză redusă în testele inițiale, poziție home compactă înainte de navigare, oprire software din interfață, verificarea spațiului înainte de execuție și testarea task-ului fără obiect înainte de testarea cu obiect. Aceste măsuri nu înlocuiesc o analiză formală de siguranță, dar reduc riscul în timpul dezvoltării.

O altă cerință nefuncțională este trasabilitatea testelor. Pentru a putea compara iterațiile, este util ca testele să fie înregistrate cu rosbag2 sau documentate prin capturi și tabele [42]. Astfel, dacă un parametru este modificat, se poate observa dacă rata de succes crește sau scade. Fără această documentare, ajustările pot deveni subiective și greu de justificat în lucrarea finală.

1.6 Analiza funcțională tehnică a sistemului integrat

Analiza funcțională tehnică are rolul de a descompune sistemul integrat în funcții și subsisteme. În loc ca sistemul să fie privit doar ca o listă de componente, el este analizat prin prisma nevoilor pe care trebuie să le îndeplinească. Nevoia principală este realizarea unui robot mobil capabil să navigheze autonom, să manipuleze un obiect și să ofere utilizatorului o interfață de vizualizare și control.

Din punct de vedere extern, sistemul trebuie să răspundă nevoii unui operator de a executa o sarcină de colectare sau plasare a unui obiect într-un mediu de laborator. Operatorul nu trebuie să controleze manual fiecare articulație, ci trebuie să poată declanșa sau urmări un scenariu în care robotul execută sarcina. Din punct de vedere intern, sistemul trebuie să coordoneze navigarea, manipularea, controlul hardware și vizualizarea.

Funcția principală a sistemului poate fi formulată astfel: realizarea unui task de pick and place autonom pe o platformă mobilă, cu feedback vizual în Unity și AR. Această funcție principală poate fi împărțită în mai multe funcții secundare, prezentate în Tabelul 1.6.1.

Tabelul 1.6.1 - Funcții tehnice ale sistemului integrat

Cod	Funcție	Subsistem responsabil	Intrări	Ieșiri
F1	Navigare autonomă	Nav2 / platformă mobilă	Goal poziție	Poziție atinsă / status
F2	Localizare	Platformă mobilă	LiDAR, odometrie, hartă	Pose robot
F3	Planificare manipulare	Movel2 / MTC	Poziție obiect, coș, stare braț	Traietorie validă
F4	Execuție manipulare	ros2_control / OMX-F	Traietorie articulații	Mișcare fizică braț
F5	Prindere obiect	Gripper OMX-F	Comandă open/close	Obiect prins / eliberat
F6	Orchestrare scenariu	Nod de integrare	Status Nav2 și MTC	Tranziții de stare
F7	Vizualizare	Unity	Topicuri ROS2	Scenă 3D actualizată
F8	Interfață AR	Unity / Meta Quest	Date scenă, input utilizator	Vizualizare MR
F9	Diagnosticare	ROS2 logs / UI	Stări noduri și topicuri	Mesaje eroare

Descompunerea în funcții arată că sistemul are trei niveluri. Primul nivel este nivelul hardware, format din platforma mobilă, brațul robotic, actuatori, senzori și componente de alimentare. Al doilea nivel este nivelul software robotic, format din ROS2, Nav2, Movel2, MTC și ros2_control. Al treilea nivel este nivelul de interacțiune, format din Unity, bridge-ul ROS2-Unity și aplicația AR.

Între aceste niveluri există interdependențe. De exemplu, dacă poziția platformei nu este suficient de precisă, brațul poate să nu ajungă la obiect. Dacă URDF-ul brațului nu este corect, Movel2 poate calcula traiectorii greșite. Dacă bridge-ul Unity primește date cu întârziere, utilizatorul poate interpreta greșit starea robotului. De aceea, analiza funcțională trebuie să includă și riscurile tehnice.

Un element central al analizei este amplasamentul brațului. În proiectarea unui robot mobil manipulator, poziția brațului nu este aleasă doar din motive mecanice, ci și din motive funcționale. Brațul trebuie să poată ajunge la obiectul de pe sol, la coșul montat pe platformă și să nu blocheze senzorii sau mișcarea platformei. Tabelul 1.6.2 prezintă variantele analizate.

Tabelul 1.6.2 - Variante de amplasament pentru brațul OMX-F pe platformă

Variantă	Avantaje	Dezavantaje	Evaluare
Montaj frontal	Acces bun la obiecte din fața robotului	Poate bloca senzori frontali și crește consola	Bun pentru pick, riscant pentru navigare
Montaj posterior	Nu blochează zona frontală	Acces dificil la obiecte din fața	Potrivit doar dacă robotul se rotește
Montaj lateral stânga	Acces la obiecte laterale	Dezechilibru lateral, workspace asimetric	Necesită analiză stabilitate
Montaj lateral dreapta	Similar lateral stânga	Dezechilibru lateral, cablare dificilă	Necesită analiză stabilitate
Montaj central-superior	Stabilitate mai bună, cablare simplă	Dificil de ajuns la sol	Bun pentru coș, mai slab pentru pick de pe jos
Montaj frontal-central	Compromis între acces și simetrie	Necesită suport mecanic robust	Variantă recomandată pentru scenariu

Pe baza criteriilor funcționale, varianta frontal-centrală este cea mai potrivită pentru scenariul propus. Aceasta permite brațului să ajungă în zona din fața platformei, unde obiectul poate fi poziționat după oprirea robotului. În același timp, montarea cât mai aproape de axa longitudinală a platformei reduce dezechilibrul lateral. Poziția exactă trebuie validată prin model 3D și prin simularea workspace-ului.

În analiza funcțională trebuie inclus și coșul. Coșul nu este un element pasiv, deoarece poziția lui influențează task-ul de place. Dacă este prea departe, brațul nu ajunge. Dacă este prea aproape de baza brațului, orientarea gripperului poate deveni dificilă. Dacă este prea înalt, brațul poate avea probleme la ridicarea obiectului peste margine. Dacă este prea jos, obiectul poate cădea sau poate lovi platforma. De aceea, poziția coșului trebuie stabilită împreună cu poziția brațului.

Un alt flux important este fluxul de date dintre ROS2 și Unity. Starea brațului și a platformei este generată în ROS2, iar Unity trebuie să o primească și să o transforme în reprezentări vizuale. Pentru aceasta, este necesară o mapare între frame-urile ROS2 și coordonatele Unity. Această mapare trebuie să fie consecventă, altfel modelul virtual nu va corespunde robotului real.

Tabelul 1.6.3 prezintă principalele riscuri tehnice identificate în etapa de proiectare.

Tabelul 1.6.3 - Riscuri tehnice și măsuri de prevenire

Risc	Cauză posibilă	Impact	Măsură de prevenire
Brațul nu ajunge la obiect	Amplasament greșit / obiect prea departe	Task eșuat	Analiză workspace și teste în simulare
Coliziune cu platforma	Scene de coliziune incompletă	Risc hardware	Modelare platformă și coș în MoveIt2
Instabilitate platformă	Masă suplimentară asimetrică	Navigare afectată	Montaj central și testare statică/dinamică
Latență Unity mare	Prea multe topicuri sau frecvență ridicată	Vizualizare nefluidă	Filtrare topicuri și limitare rate
Incompatibilitate software	Versiuni ROS2 / Unity / pachete	Blocaj implementare	Testare incrementală și documentare versiuni
Task MTC instabil	Parametri geometrici nepotriviti	Rată de succes redusă	Ajustare stagii și validare pe hardware
Calibrare AR greșită	Aliniere real-virtual incompletă	Utilizator indus în eroare	Procedură de calibrare și marcaje vizuale

Analiza funcțională arată că sistemul trebuie proiectat ca un ansamblu coerent. Nu este suficient ca fiecare componentă să funcționeze separat.

Navigarea trebuie să aducă robotul într-o poziție utilă pentru manipulare, manipularea trebuie să țină cont de platformă și coș, iar interfața trebuie să afișeze corect starea subsistemelor. Această concluzie determină arhitectura hardware și software prezentată în subcapitolele următoare.

Analiza funcțională poate fi privită și din perspectiva procesului de proiectare inginerească. În literatura de proiectare, o soluție tehnică este dezvoltată prin identificarea nevoii, descompunerea funcțională, generarea variantelor, evaluarea lor și alegerea soluției finale [48], [49]. În cazul de față, nevoia este realizarea unui sistem de manipulare mobilă cu interfață AR. Variantele apar în amplasamentul brațului, poziția coșului, modul de alimentare, modul de comunicare ROS2-Unity și nivelul de control oferit în AR.

Pentru a evalua aceste variante, se pot folosi criterii ponderate. De exemplu, pentru amplasamentul brațului, criteriile cele mai importante sunt accesul la obiect, accesul la coș, stabilitatea platformei, simplitatea mecanică și impactul asupra senzorilor. Fiecare variantă poate primi un scor, iar soluția cu cel mai bun compromis poate fi aleasă. Această abordare face decizia mai transparentă și mai ușor de justificat în lucrare.

Tabelul 1.6.4 - Matrice orientativă de evaluare a amplasamentului brațului

Criteriu	Pondere	Frontal	Posterior	Lateral	Frontal-central
Acces la obiect	0.30	5	2	3	5
Acces la coș	0.20	4	3	3	4
Stabilitate	0.20	3	4	2	4
Impact asupra senzorilor	0.15	2	5	4	3
Simplitate cablare	0.15	3	4	3	4
Scor ponderat	1.00	3.65	3.35	3.00	4.15

Valorile din tabel sunt orientative și trebuie validate prin modelare și testare. Totuși, ele arată logica deciziei: varianta frontal-centrală oferă un compromis bun între accesul la obiect și stabilitate. Varianta frontală simplă poate fi foarte bună pentru pick, dar riscă să afecteze senzorii frontali. Varianta laterală poate părea convenabilă mecanic, dar produce asimetrie și poate reduce stabilitatea.

În analiza funcțională trebuie luată în calcul și planificarea mișcării la nivel conceptual. Conform principiilor generale din robotică, un sistem de manipulare trebuie să gestioneze percepția, planificarea și execuția [50], [51]. În lucrarea de față, percepția completă a obiectului nu este obiectivul principal, dar poziția obiectului trebuie totuși definită. Planificarea este realizată cu MoveIt2/MTC, iar execuția prin ros2_control și actuatorii Dynamixel.

Din punct de vedere al planificării, task-ul poate fi văzut ca o problemă de căutare într-un spațiu de configurații. Brațul trebuie să găsească o cale de la configurația curentă la configurația de prindere, apoi la configurația de plasare. Această formulare este apropiată de problemele de planning din robotică, unde obstacolele și limitele articulațiilor restrâng spațiul soluțiilor [53]. Într-un sistem real, acest planning trebuie combinat cu execuția robustă și cu feedback-ul de stare.

1.7 Proiectarea hardware a integrării

Proiectarea hardware urmărește modul în care brațul OMX-F, coșul, alimentarea și eventualele componente auxiliare sunt integrate pe platforma mobilă. Chiar dacă platforma mobilă este existentă și brațul este un kit comercial, integrarea fizică dintre ele trebuie proiectată cu atenție. O montare greșită poate afecta stabilitatea, workspace-ul, cablarea și siguranța.

Primul element de proiectare este suportul mecanic al brațului. Acesta trebuie să fixeze baza brațului pe șasiul platformei, să reziste la eforturile produse în timpul mișcării și să păstreze poziția brațului constantă. Dacă suportul flexează sau se deplasează, modelul geometric din URDF nu mai corespunde realității, iar planificarea devine mai puțin precisă. Din acest motiv, suportul trebuie realizat dintr-un material suficient de rigid și trebuie prins în puncte solide ale șasiului.

Al doilea element este poziția brațului față de platformă. Varianta recomandată este montarea frontal-centrală sau cât mai aproape de axa longitudinală, astfel încât brațul să poată ajunge la obiectele din fața robotului. Poziția finală trebuie stabilită printr-un studiu de amplasament în model 3D, verificând dacă brațul poate atinge atât zona de pick, cât și zona de place. Pentru această etapă pot fi utilizate instrumente CAD precum Onshape sau un model URDF verificat în RViz și Gazebo [31], [33], [39].

Al treilea element este coșul. Coșul trebuie să fie montat într-o poziție accesibilă brațului și stabilă în timpul deplasării. Dimensiunile coșului trebuie alese în funcție de obiectele manipulate și de spațiul disponibil pe platformă. Un coș prea mare poate bloca mișcarea brațului sau poate afecta navigarea. Un coș prea mic poate face task-ul de place dificil, deoarece obiectul trebuie poziționat cu precizie mai mare.

Tabelul 1.7.1 prezintă elementele hardware care trebuie proiectate sau adaptate.

Tabelul 1.7.1 - Elemente hardware ale integrării

Element	Rol	Cerințe principale	Observații
Suport braț	Fixarea OMX-F pe platformă	Rigiditate, aliniere, acces la șuruburi	Contribuție de integrare
Poziție braț	Definirea workspace-ului	Acces la obiect și coș	Rezultă din studiul de amplasament
Coș	Zonă de place	Stabil, accesibil, dimensiune potrivită	Trebuie inclus în scena de coliziune
Cablare	Alimentare și comunicație	Fără tensionare în timpul mișcării	Traseu protejat
Alimentare	Energie pentru actuatori	Tensiune și curent compatibile	Verificare separată de platformă
Elemente siguranță	Evitare coliziuni fizice	Margini protejate, prinderi ferme	Necesare la testare

Din punct de vedere electric, brațul are nevoie de alimentare stabilă pentru actuatori. Servomotoarele Dynamixel pot consuma curent variabil în funcție de sarcină și mișcare [23]. Dacă alimentarea este insuficientă, pot apărea resetări, scăderi de tensiune sau comportament instabil. De aceea, soluția de alimentare trebuie dimensionată în funcție de consumul maxim estimat și trebuie verificată înainte de execuția task-ului pe robot.

O decizie importantă este dacă brațul este alimentat din sursa platformei sau dintr-o sursă separată. Alimentarea din sursa platformei reduce numărul de baterii și poate simplifica exploatarea, dar crește sarcina pe sistemul electric existent. Alimentarea separată izolează brațul de platformă, dar adaugă masă și necesită gestionarea unui acumulator suplimentar. În faza de proiectare, varianta recomandată este folosirea unei alimentări dedicate pentru braț sau a unui modul de conversie verificat, astfel încât fluctuațiile produse de actuatori să nu afecteze platforma.

Tabelul 1.7.2 prezintă o analiză orientativă pentru alimentarea brațului și a componentelor auxiliare. Valorile trebuie completate sau actualizate în etapa de implementare cu măsurători reale.

Tabelul 1.7.2 - Estimare orientativă a consumului pentru subsistemul de manipulare

Componentă	Tensiune nominală	Consum estimat	Observații
Actuatori Dynamixel	conform documentației	variabil, dependent de sarcină	Consum maxim la ridicare obiect
Controller OpenRB-150	USB / alimentare dedicată	reduc	Necesită comunicație stabilă
Cameră auxiliară	USB 5 V	reduc-mediu	Opțională pentru vizualizare
Convertor DC-DC	intrare platformă / baterie	dependent de sarcină	Trebuie dimensionat cu rezervă
Meta Quest	baterie proprie	separat	Nu încarcă platforma
Laptop / stație Unity	alimentare proprie	separat	Folosit pentru dezvoltare și testare

În proiectarea hardware trebuie luată în calcul și cablarea. Cablurile trebuie să permită mișcarea completă a brațului fără întindere, blocare sau contact cu roțile platformei. De asemenea, cablurile nu trebuie să intre în spațiul de lucru al gripperului. O soluție practică este ghidarea cablurilor pe lângă suportul brațului și fixarea lor cu cleme, lăsând o rezervă de lungime pentru articulațiile mobile.

Pentru integrarea coșului, poziția recomandată este în zona accesibilă brațului după prinderea obiectului, dar fără să fie nevoie de o mișcare foarte amplă. Dacă obiectul este preluat din fața platformei, coșul poate fi amplasat în partea superioară sau ușor în spatele brațului, astfel încât traiectoria de la pick la place să fie scurtă. Totuși, coșul nu trebuie să blocheze mișcarea brațului când acesta revine în poziția home.

Un alt element important este definirea pozițiilor sigure. Brațul trebuie să aibă o poziție home în care nu depășește excesiv conturul platformei și nu riscă să lovească obstacole în timpul navigării. Înainte de deplasarea platformei, brațul ar trebui să revină într-o poziție compactă. Această cerință influențează atât proiectarea hardware, cât și proiectarea software a scenariului integrat.

În concluzie, proiectarea hardware a integrării urmărește să creeze o configurație fizică stabilă, accesibilă și compatibilă cu task-ul de pick and place. Suportul brațului, poziția coșului, alimentarea și cablarea sunt elemente care trebuie stabilite înainte de implementarea software finală, deoarece ele determină limitele reale ale sistemului.

1.8 Proiectarea software a sistemului integrat

Arhitectura software reprezintă componenta care leagă toate elementele sistemului. Platforma mobilă, brațul robotic, planificarea MTC, Unity și AR trebuie

să comunice prin interfețe clare. În ecosistemul ROS2, această comunicare se realizează prin noduri, topicuri, servicii, acțiuni și transformări [10], [11]. Proiectarea software trebuie să stabilească ce modul produce date, ce modul le consumă și cum sunt gestionate stările scenariului.

În lucrarea de față, arhitectura software este gândită modular. Fiecare modul are o responsabilitate clară: navigare, manipulare, orchestrare, vizualizare, AR și diagnosticare. Această separare este importantă deoarece permite testarea individuală a componentelor. De exemplu, task-ul MTC poate fi testat fără Unity, bridge-ul Unity poate fi testat cu topicuri simulate, iar navigarea Nav2 poate fi testată fără execuția brațului.

1.8.1 Arhitectura modulară a sistemului

Arhitectura modulară propusă include următoarele module: modulul de navigare, modulul de manipulare, modulul de orchestrare, modulul bridge ROS2-Unity, modulul AR și modulul de diagnosticare. Tabelul 1.8.1 prezintă rolul fiecărui modul.

Tabelul 1.8.1 - Module software ale sistemului integrat

Modul	Tehnologii principale	Rol	Interfețe principale
Navigare	Nav2, ROS2	Deplasarea platformei către POI	NavigateToPose, /odom, /tf
Manipulare	Movel2, MTC, ros2_control	Execuția task-ului pick and place	/joint_states, acțiuni control braț
Orchestrare	Nod ROS2 Python/C++	Coordonarea Nav2 și MTC	status, trigger, acțiuni
Bridge Unity	ROS-TCP-Connector	Transfer date ROS2-Unity	topicuri stare și comenzi
AR	Unity, Meta Quest SDK	Vizualizare realitate mixtă	obiecte 3D, input XR
Diagnosticare	RViz, logs, rosbag2	Monitorizare și testare	loguri, topicuri, înregistrări

Modulul de navigare este considerat existent, dar trebuie integrat în scenariul complet. El primește un goal de tip NavigateToPose și aduce platforma în zona de lucru [12]. După atingerea punctului de interes, modulul de orchestrare poate declanșa task-ul de manipulare. Această tranziție trebuie realizată doar după ce platforma este oprită și poziția este considerată stabilă.

Modulul de manipulare conține modelul Movel2, task-ul MTC și interfața cu controllerele brațului. El primește parametrii task-ului, cum ar fi poziția obiectului și poziția coșului, și încearcă să calculeze o soluție. Dacă soluția este găsită, traiectoria este executată pe braț. Dacă planificarea eșuează, modulul trebuie să raporteze motivul sau cel puțin stagiul în care a apărut problema.

Modulul de orchestrare este cel care transformă componentele separate într-un scenariu unitar. El funcționează ca o mașină de stări. Stările principale pot fi: Idle, NavigateToObject, WaitForStabilization, ExecutePickPlace, VerifyTask, NavigateNext și Error. Această mașină de stări este importantă deoarece previne situații în care platforma se mișcă în timp ce brațul execută task-ul sau în care task-ul pornește înainte ca robotul să fie poziționat.

Modulul bridge ROS2-Unity transmite date către Unity și primește comenzi din interfață. În faza de proiectare, nu este recomandat ca Unity să controleze direct fiecare articulație a brațului. Controlul de nivel jos rămâne în ROS2, iar Unity oferă comenzi de nivel mai înalt, cum ar fi pornirea scenariului sau oprirea

de urgență software. Această separare este mai sigură și mai ușor de verificat.

1.8.2 Proiectarea task-ului pick and place cu MoveIt2 Task Constructor

Task-ul pick and place este componenta centrală a manipulării. În proiectarea lui trebuie definite obiectul, poziția de prindere, poziția de plasare, traiectoriile intermediare și condițiile de succes. Pentru a păstra structura clară, task-ul este împărțit în stagiile prezentate anterior în Tabelul 1.3.2.

Parametrii principali ai task-ului sunt prezentați în Tabelul 1.8.2.

Tabelul 1.8.2 - Parametri principali pentru task-ul MTC

Parametru	Descriere	Valoare proiectată / observație
object_frame	Frame-ul obiectului	Raportat la baza platformei sau la hartă
grasp_pose	Poziția de prindere	Stabilită în funcție de dimensiunea obiectului
pre_grasp_offset	Distanța de apropiere	Permite orientarea gripperului înainte de prindere
lift_height	Înălțimea de ridicare	Suficientă pentru a evita solul
place_pose	Poziția în coș	Raportată la frame-ul coșului
retreat_offset	Mișcare după eliberare	Evită coliziunea cu coșul
home_configuration	Poziție sigură braț	Folosită înainte și după navigare
planning_time	Timp maxim de planificare	Ajustat în testare
collision_objects	Platformă, coș, obiect	Definite în scena MoveIt2

Poziția obiectului poate fi stabilită inițial manual, pentru a reduce complexitatea. Într-o etapă viitoare, aceasta poate fi obținută prin percepție vizuală sau prin interfața AR. Pentru această lucrare, obiectivul principal este integrarea task-ului de manipulare, nu dezvoltarea unui sistem complet de detecție a obiectelor. Această delimitare este importantă deoarece permite concentrarea pe planificare și integrare.

Poziția de place este definită în interiorul coșului. Pentru a evita coliziunile cu marginile, task-ul poate include o poziție deasupra coșului, urmată de o coborâre controlată. După eliberarea obiectului, brațul se retrage și revine în poziția home. Această secvență reduce riscul ca gripperul să lovească coșul.

1.8.3 Proiectarea ROS2-Unity bridge

Bridge-ul ROS2-Unity este proiectat ca un modul de vizualizare și control de nivel înalt. În Unity se importă modele 3D pentru platformă, braț, coș și obiect. Poziția platformei poate fi actualizată pe baza odometriei sau a transformărilor, iar poziția brațului pe baza topicului /joint_states. Pentru ca mișcarea să fie fluidă, Unity trebuie să interpoleze sau să actualizeze pozițiile la o frecvență compatibilă cu frame rate-ul aplicației.

Un aspect important este diferența dintre sistemele de coordonate. ROS2 folosește în mod obișnuit convenția X înainte, Y stânga, Z sus, în timp ce Unity folosește propriul sistem de coordonate. De aceea, bridge-ul trebuie să includă o transformare între coordonate. Dacă această transformare nu este corectă, robotul poate apărea rotit sau deplasat greșit în scenă.

În Unity, informațiile sunt organizate în trei categorii: reprezentare 3D, panouri de stare și comenzi. Reprezentarea 3D include modelul robotului și al

brațului. Panourile de stare includ conexiunea ROS2, statusul Nav2, statusul MTC și eventualele erori. Comenzile includ pornirea scenariului, oprirea demonstrației și comenzi simple de test.

1.8.4 Proiectarea scenariului integrat Nav2 și MTC

Scenariul integrat este proiectat ca o succesiune de stări. Platforma primește un goal de navigare și se deplasează către punctul de interes. După atingerea goal-ului, robotul așteaptă un timp scurt pentru stabilizare. Apoi este declanșat task-ul MTC. Dacă task-ul reușește, brațul revine în poziția home, iar platforma poate primi un nou goal. Dacă task-ul eșuează, sistemul intră într-o stare de eroare și raportează problema în Unity.

Tabelul 1.8.3 prezintă mașina de stări proiectată.

Tabelul 1.8.3 - Mașina de stări pentru scenariul Nav2 + MTC

Stare	Descriere	Condiție de tranziție	Stare următoare
Idle	Sistem pregătit	Comandă start	NavigateToPOI
NavigateToPOI	Platforma navighează	Goal atins	Stabilize
Stabilize	Robotul așteaptă oprirea	Timp de stabilizare trecut	ExecuteMTC
ExecuteMTC	Brațul execută pick and place	Task reușit	ReturnHome
ReturnHome	Brațul revine în poziție sigură	Poziție atinsă	NavigateNext
NavigateNext	Platforma continuă scenariul	Goal finalizat	Idle
Error	Sistem în eroare	Reset / diagnosticare	Idle

Această mașină de stări permite controlul clar al scenariului. Ea previne suprapunerea acțiunilor și oferă un mod simplu de diagnosticare. De exemplu, dacă sistemul se oprește în ExecuteMTC, problema este legată de manipulare. Dacă se oprește în NavigateToPOI, problema este legată de navigare. Dacă se oprește în Stabilize, problema poate fi legată de poziționare sau de feedback-ul de stare.

Nav2 folosește în mod obișnuit concepte de behavior trees pentru organizarea comportamentului de navigare [41]. În lucrarea de față nu este necesară modificarea arborelui de comportament existent al Nav2, însă ideea de comportamente secvențiale este utilă și pentru orchestrarea sistemului. Practic, scenariul complet poate fi privit ca un arbore sau o mașină de stări la nivel superior: navighează, verifică, manipulează, verifică, continuă. Această separare ajută la menținerea stack-ului Nav2 stabil și la adăugarea manipulării ca modul suplimentar.

Pentru sincronizarea datelor, sistemul folosește transformările tf2 și topicurile ROS2 [43]. Transformările sunt importante deoarece poziția obiectului, poziția coșului, poziția brațului și poziția platformei trebuie exprimate în frame-uri compatibile. Dacă obiectul este definit în frame-ul hărții, iar task-ul MTC are nevoie de poziția lui în frame-ul bazei brațului, transformarea trebuie calculată corect. O eroare în această transformare poate duce la o prindere greșită.

În cazul în care sistemul va utiliza imagini sau date sincronizate de la mai mulți senzori, pachetul message_filters poate fi folosit pentru sincronizarea mesajelor [45]. Pentru versiunea curentă, poziția obiectului poate fi definită manual, dar arhitectura trebuie să permită o extindere ulterioară către percepție. Din acest motiv, este util ca nodurile să fie proiectate astfel încât să accepte date

cu timestamp și frame_id corecte.

Tabelul 1.8.5 - Mesaje și transformări critice în scenariul integrat

Element	Tip date	Frame / referință	Motiv
Poziție platformă	Odometry / TF	map, odom, base_link	Necesare pentru navigare și Unity
Stare braț	JointState	articulații OMX-F	Necesare pentru MoveIt2 și vizualizare
Bază braț	TF static	base_link -> omx_base	Definește montajul fizic
Obiect	PoseStamped	base_link sau map	Intrare pentru MTC
Coș	Collision object / TF	frame platformă	Țintă pentru place
Status task	Mesaj custom / std_msgs	logic	Feedback în Unity
Comenzi Unity	Twist / trigger	ROS2	Control de nivel înalt

Pentru testare și depanare, rosbag2 poate fi folosit pentru înregistrarea topicurilor importante [42]. De exemplu, se pot înregistra /joint_states, /tf, statusul MTC, odometria și mesajele trimise către Unity. Dacă un test eșuează, aceste date pot fi analizate ulterior pentru a vedea dacă problema a fost de planificare, comunicație sau execuție.

1.8.5 Arhitectura sistemului AR cu Meta Quest

Arhitectura AR este construită peste scena Unity. Meta Quest rulează aplicația de realitate mixtă, Unity gestionează obiectele virtuale, iar ROS2 transmite datele robotului. Fluxul general este: ROS2 Jazzy -> ROS-TCP-Connector -> Unity -> Meta Quest. În sens invers, inputurile utilizatorului din AR pot fi transmise către Unity și apoi către ROS2, dar în această lucrare controlul direct este limitat.

Tabelul 1.8.4 prezintă componentele arhitecturii AR.

Tabelul 1.8.4 - Componentele arhitecturii AR

Componentă	Rol	Observații
Meta Quest	Dispozitiv de vizualizare MR	Rulează aplicația Unity
Unity Scene	Mediu 3D și UI	Conține robotul, brațul, coșul, panourile
ROS-TCP-Connector	Client Unity pentru ROS	Primește și trimite mesaje
ROS-TCP-Endpoint	Server în partea ROS2	Interfață de comunicație
ROS2 Jazzy	Middleware robotic	Sursă de date și control
MTC Status Publisher	Nod de stare task	Trimite stagiul curent în UI
AR Input Manager	Gestionare input utilizator	Direcție pentru dezvoltări viitoare

Pentru demonstrația parțială, obiectivul este vizualizarea sistemului în realitate mixtă. Utilizatorul poate vedea modelul robotului și starea acestuia, fără ca AR să controleze complet brațul. Această limitare este justificată deoarece controlul prin AR necesită validări suplimentare de siguranță și calibrare.

1.8.6 Instrumente și tehnologii software utilizate

Instrumentele software utilizate sunt alese pentru compatibilitate cu ecosistemul ROS2 și pentru suportul oferit în robotică. Python poate fi folosit pentru noduri ROS2 de orchestrare, deoarece permite dezvoltare rapidă și are suport bun în ROS2. C# este folosit în Unity pentru logica interfeței și pentru integrarea cu pachetele ROS-TCP. ROS2 Jazzy este baza de comunicație a

sistemului, Nav2 este folosit pentru navigare, MoveIt2 și MTC pentru manipulare, Gazebo pentru simulare, Unity pentru vizualizare și Meta Quest pentru AR [11], [12], [13], [14], [15], [16], [17], [18].

Docker poate fi folosit pentru izolarea mediilor software atunci când apar incompatibilități între versiuni sau dependențe [38]. În proiecte robotice, această izolare este utilă deoarece aceeași aplicație poate depinde de versiuni specifice de ROS2, Python, biblioteci de control sau pachete Unity. Totuși, pentru deployment pe robot, trebuie verificat dacă rularea în container nu introduce probleme de acces la hardware.

RViz este utilizat pentru verificarea transformărilor, a modelului robotului și a stării MoveIt2 [33]. Gazebo este utilizat pentru simulare și validare înainte de testarea fizică [18], [44]. Unity este utilizat pentru reprezentarea vizuală și pentru interfața AR. Împreună, aceste instrumente acoperă întregul ciclu de dezvoltare: modelare, simulare, planificare, execuție și vizualizare.

1.9 Simulare în Gazebo și MoveIt2

Simularea este o etapă importantă în dezvoltarea sistemului deoarece permite verificarea modelului înainte de testarea pe hardware fizic. Într-un sistem cu braț robotic, o eroare de modelare poate produce mișcări incorecte, coliziuni sau imposibilitatea planificării. Prin simulare, aceste probleme pot fi identificate mai devreme și cu risc mai mic.

Gazebo este utilizat pentru simularea mediului și a robotului, iar MoveIt2 pentru planificarea mișcărilor [18], [44]. În prima etapă, se verifică modelul URDF al brațului. Link-urile trebuie să apară corect, articulațiile trebuie să se miște în limitele așteptate, iar transformările trebuie să fie coerente. În a doua etapă, se adaugă platforma, coșul și obiectul. În a treia etapă, se testează task-ul MTC.

Tabelul 1.9.1 prezintă testele propuse în simulare.

Tabelul 1.9.1 - Teste propuse în Gazebo și MoveIt2

Test	Scop	Criteriu de succes
Verificare URDF	Confirmarea modelului geometric	Link-uri și articulații afișate corect
Verificare TF	Confirmarea transformărilor	Frame-uri coerente în RViz
Mișcare articulații	Testarea limitelor	Articulații se mișcă fără erori
Planificare simplă	Move to pose	Traietorie validă calculată
Scenă coliziune	Platformă și coș	Coliziunile sunt detectate corect
Stagiu pre-grasp	Apropiere de obiect	Poziția este accesibilă
Stagiu pick	Prindere și ridicare	Obiect ridicat fără coliziuni
Stagiu place	Plasare în coș	Obiect eliberat în zona țintă
Bridge Unity simulat	Vizualizare mișcare	Scena Unity se actualizează

Simularea nu trebuie tratată ca o garanție absolută. În mediul real pot apărea jocuri mecanice, erori de calibrare, diferențe de frecare, întârzieri de comunicație și limitări ale actuatorilor. Totuși, simularea este foarte utilă pentru validarea logicii și pentru reducerea numărului de încercări riscante pe hardware.

Un aspect important este compararea comportamentului simulat cu cel fizic. După ce task-ul funcționează în simulare, aceiași parametri trebuie testați pe brațul real. Dacă apar diferențe, ele trebuie documentate. De exemplu, poziția de

grasp poate trebui ajustată cu câțiva milimetri, înălțimea de lift poate trebui mărită, iar viteza mișcării poate trebui redusă pentru stabilitate.

În concluzie, simularea este folosită ca etapă de validare intermediară. Ea ajută la verificarea modelului și a task-ului, dar validarea finală se face pe sistemul fizic integrat.

Pentru ca simularea să fie utilă, scenariile de test trebuie definite înainte de implementare. Dacă simularea este folosită doar pentru a vedea robotul mișcându-se, valoarea ei este limitată. În schimb, dacă sunt definite criterii clare, simularea devine un instrument de validare. De exemplu, se poate verifica dacă brațul ajunge la obiect din trei poziții diferite, dacă poate plasa obiectul în coș fără coliziuni și dacă revine în poziția home.

Tabelul 1.9.2 - Scenarii de simulare propuse

Scenariu	Configurație	Ce se verifică	Rezultat așteptat
S1	Obiect în poziție nominală	Task complet pick and place	Succes fără coliziuni
S2	Obiect deplasat stânga/dreapta	Toleranța la variații laterale	Succes în limite definite
S3	Obiect deplasat înainte/înapoi	Limita workspace-ului	Identificarea zonei accesibile
S4	Coș în poziție nominală	Traectorie place	Obiect plasat corect
S5	Coș cu margini modelate	Coliziuni la place	Traectorie evită marginile
S6	Braț revine home	Siguranță pentru navigare	Configurație compactă
S7	Bridge Unity activ	Sincronizare vizuală	Modelul urmărește robotul
S8	Eșec planificare intenționat	Raportare eroare	Starea este afișată în UI

Un avantaj al simulării este că permite generarea unor situații care ar fi incomode sau riscante pe robotul real. De exemplu, se poate testa ce se întâmplă dacă obiectul este în afara workspace-ului sau dacă coșul este poziționat greșit. Sistemul ar trebui să eșueze controlat, nu să genereze o mișcare periculoasă. Această diferență este importantă: un eșec controlat este acceptabil în proiectare, în timp ce un eșec necontrolat poate indica o problemă de siguranță.

În etapa de simulare trebuie urmărită și diferența dintre coliziuni vizuale și coliziuni fizice. Modelele de coliziune pot fi mai simple decât modelele vizuale, pentru a reduce complexitatea. Totuși, dacă sunt prea simplificate, pot permite traiectorii care în realitate ar lovi robotul. Dacă sunt prea mari, pot bloca planificarea. Alegerea geometriei de coliziune este astfel un compromis între acuratețe și eficiență.

Datele obținute în simulare pot fi folosite pentru a construi tabele de rezultate în Capitolul 3. De exemplu, se pot nota timpii de planificare, numărul de soluții găsite și stagiile care eșuează. Înregistrarea cu rosbag2 a testelor simulate poate ajuta la compararea cu testele fizice [42]. Astfel, simularea nu rămâne doar o etapă vizuală, ci devine parte din metodologia de validare.

1.10 Costul proiectului

Estimarea costului proiectului are rolul de a evidenția resursele utilizate și efortul necesar pentru realizarea sistemului. Deoarece lucrarea pornește de la o platformă mobilă existentă și de la un braț pus la dispoziție, costul total nu reflectă achiziția completă a unui sistem de la zero, ci costurile asociate resurselor

utilizate și integrării.

Tabelul 1.10.1 prezintă o estimare orientativă a resurselor hardware. Valorile pot fi actualizate în lucrarea finală în funcție de prețurile reale și de componentele efectiv utilizate.

Tabelul 1.10.1 - Estimare orientativă a costurilor hardware

Resursă	Rol	Modalitate de procurare	Cost estimat
Platformă mobilă Xplorer-A	Bază mobilă autonomă	Existentă în laborator	0 RON în proiect
Braț ROBOTIS OMX-F	Manipulare obiecte	Pus la dispoziție	conform achiziției
Elemente suport mecanic	Fixare braț pe platformă	Realizare / achiziție	100-300 RON
Coș și elemente fixare	Zonă de place	Achiziție / printare 3D	50-150 RON
Convertor / alimentare	Alimentare braț	Achiziție dacă este necesar	100-250 RON
Cabluri și conectori	Interconectare	Achiziție	50-150 RON
Meta Quest 3	Vizualizare AR	Disponibil utilizator	0 RON în proiect
Laptop / PC dezvoltare	Unity, ROS2, testare	Disponibil	0 RON în proiect

Din punct de vedere software, majoritatea tehnologiilor folosite sunt open-source sau gratuite pentru uz educațional. ROS2, Nav2, MoveIt2, Gazebo, RViz și pachetele ROBOTIS pot fi folosite fără costuri de licență [11], [12], [13], [18], [29]. Unity poate fi utilizat în varianta gratuită pentru proiecte educaționale, în condițiile respectării licenței [16]. Această situație reduce costul financiar al proiectului, dar nu reduce complexitatea tehnică.

Tabelul 1.10.2 prezintă resursele software.

Tabelul 1.10.2 - Resurse software utilizate

Resursă software	Rol	Cost licență	Observații
ROS2 Jazzy	Middleware robotic	0 RON	Open-source
Nav2	Navigare autonomă	0 RON	Integrat pe platformă
MoveIt2	Planificare mișcare	0 RON	Open-source
MoveIt2 Task Constructor	Task pick and place	0 RON	Extensie MoveIt2
Gazebo	Simulare	0 RON	Open-source
RViz	Vizualizare ROS2	0 RON	Open-source
Unity	Interfață 3D / AR	0 RON educațional	Necesită configurare proiect
ROS-TCP-Connector	Bridge ROS-Unity	0 RON	Repository Unity
Meta Quest SDK	AR / MR	0 RON	Conform condițiilor Meta
Docker	Containerizare	0 RON pentru utilizare de bază	Util în dezvoltare

Pe lângă costurile materiale, este importantă estimarea volumului de muncă. Proiectul implică analiză, proiectare, integrare, configurare software, simulare, testare și documentare. Tabelul 1.10.3 prezintă o estimare orientativă a activităților principale.

Tabelul 1.10.3 - Estimarea volumului de muncă

Activitate	Descriere	Timp estimat
Documentare tehnică	Studierea OMX-F, ROS2, MoveIt2, Unity, AR	20 h
Analiză amplasament	Variante poziție braț și coș	12 h

Activitate	Descriere	Timp estimat
Proiectare hardware	Suport, alimentare, cablare	15 h
Configurare software	ROS2, Movelt2, pachete ROBOTIS	20 h
Proiectare task MTC	Stagii pick and place și parametri	18 h
Simulare Gazebo / RViz	Validare model și task	15 h
Bridge ROS2-Unity	Configurare comunicație și scenă	20 h
Componentă AR	Adaptare Unity pentru Meta Quest	15 h
Integrare Nav2-MTC	Mașină de stări și scenariu	18 h
Testare și ajustare	Teste pe componente și sistem	25 h
Documentare finală	Figuri, tabele, rezultate	20 h
Total estimat		198 h

Estimarea volumului de muncă arată că partea cea mai consumatoare de timp nu este o singură componentă, ci integrarea dintre componente. Fiecare subsistem are documentație proprie, dar problemele apar la interfețe: ROS2 cu Unity, Movelt2 cu hardware, MTC cu scena de coliziune, Nav2 cu orchestrarea task-ului și AR cu reprezentarea real-virtuală.

În concluzie, costul financiar al proiectului este redus prin folosirea resurselor existente și a software-ului open-source, dar costul tehnic este semnificativ prin efortul de integrare. Această observație este importantă deoarece arată că valoarea proiectului nu stă în achiziția componentelor, ci în modul în care ele sunt puse să funcționeze împreună.

1.11 Concluzii capitol

Capitolul 1 a prezentat proiectarea sistemului integrat de manipulare autonomă și interfață AR. Au fost analizate brațul robotic OMX-F, cadrul Movelt2 și Movelt2 Task Constructor, arhitectura ROS2-Unity, cerințele și restricțiile de proiectare, analiza funcțională, integrarea hardware, arhitectura software, simularea și costurile proiectului.

Brațul OMX-F a fost ales deoarece oferă o platformă compactă și compatibilă cu ROS2 pentru realizarea unui task de pick and place. Deși este un kit comercial, integrarea sa pe o platformă mobilă ridică probleme reale de proiectare: amplasament, stabilitate, alimentare, cablare și definirea corectă a modelului geometric.

Movelt2 și Movelt2 Task Constructor reprezintă baza pentru manipulare. Movelt2 oferă planificarea mișcării și verificarea coliziunilor, iar MTC permite organizarea task-ului în stagii clare. Această structură este potrivită pentru scenariul propus, deoarece fiecare etapă a task-ului poate fi analizată și ajustată separat.

Componenta ROS2-Unity și extensia AR adaugă sistemului o interfață mai accesibilă. Unity permite vizualizarea robotului într-o scenă 3D, iar Meta Quest permite demonstrarea unei interfețe de realitate mixtă. Această componentă nu înlocuiește instrumentele tehnice precum RViz, ci le completează printr-un mod de prezentare mai intuitiv pentru utilizator.

Analiza cerințelor și restricțiilor a arătat că principalele riscuri sunt legate de workspace-ul brațului, poziția coșului, stabilitatea platformei, compatibilitatea

software și latența interfeței. Pentru reducerea acestor riscuri, capitolul a propus o abordare incrementală: testare separată, simulare, integrare treptată și validare pe hardware fizic.

În final, proiectarea sistemului arată că lucrarea nu este doar o aplicație izolată de manipulare, ci un sistem integrat în care navigarea autonomă, planificarea mișcării, execuția hardware și interacțiunea om-robot trebuie să funcționeze împreună. Capitolul următor poate porni de la această arhitectură pentru a descrie realizarea și implementarea efectivă a sistemului.

Bibliografie suplimentară pentru Capitolul 1

[29] ROBOTIS-GIT. (2025). open_manipulator - GitHub Repository.

https://github.com/ROBOTIS-GIT/open_manipulator

[30] ros2_control. (2025). Controller Manager and ros2_control Documentation.

<https://control.ros.org/>

[31] ROS Wiki. (2024). URDF XML Documentation. <https://wiki.ros.org/urdf/XML>

[32] MoveIt. (2024). Setup Assistant and SRDF Configuration.

https://moveit.picknik.ai/main/doc/examples/setup_assistant/setup_assistant_tutorial.html

[33] Open Robotics. (2024). RViz User Guide. [https://docs.ros.org/en/rolling/Tutorials/Intermediate/R](https://docs.ros.org/en/rolling/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html)

[Viz/RViz-User-Guide/RViz-User-Guide.html](https://docs.ros.org/en/rolling/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html)

[34] Unity Technologies. (2024). ROS-TCP-Endpoint - GitHub Repository.

<https://github.com/Unity-Technologies/ROS-TCP-Endpoint>

[35] Khronos Group. (2024). OpenXR Overview. <https://www.khronos.org/openxr/>

[36] Unity Technologies. (2024). XR Interaction Toolkit Documentation.

<https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@latest>

[37] Meta. (2024). Meta XR Core SDK Documentation.

<https://developer.oculus.com/documentation/unity/unity-core-sdk/>

[38] Docker. (2024). Docker Documentation. <https://docs.docker.com/>

[39] Rhoban. (2024). onshape-to-robot - Export robot models from Onshape to URDF.

<https://github.com/Rhoban/onshape-to-robot>

[40] Open Motion Planning Library. (2024). OMPL Documentation. <https://ompl.kavrakilab.org/>

[41] BehaviorTree.CPP. (2024). BehaviorTree.CPP Documentation. <https://www.behaviortree.dev/>

[42] Open Robotics. (2024). rosbag2 Documentation. <https://github.com/ros2/rosbag2>

[43] ROS 2. (2024). tf2 Tutorials and Concepts. [https://docs.ros.org/en/rolling/Tutorials/Intermediate/](https://docs.ros.org/en/rolling/Tutorials/Intermediate/Tf2/Tf2-Main.html)

[Tf2/Tf2-Main.html](https://docs.ros.org/en/rolling/Tutorials/Intermediate/Tf2/Tf2-Main.html)

[44] Gazebo. (2024). ros2_control integration with Gazebo.

https://control.ros.org/rolling/doc/gazebo_ros2_control/doc/index.html

[45] ROS 2. (2024). message_filters Documentation. https://docs.ros.org/en/rolling/p/message_filters/

[46] ISO. (2011). ISO 10218-1: Robots and robotic devices - Safety requirements for industrial robots - Part 1: Robots. International Organization for Standardization.

[47] ISO. (2016). ISO/TS 15066: Robots and robotic devices - Collaborative robots. International Organization for Standardization.

[48] Pahl, G., Beitz, W., Feldhusen, J., & Grote, K.-H. (2007). Engineering Design: A Systematic Approach. Springer.

[49] Ullman, D. G. (2010). The Mechanical Design Process. McGraw-Hill.

[50] Siciliano, B., & Khatib, O. (Eds.). (2016). Springer Handbook of Robotics. Springer.

[51] Craig, J. J. (2018). Introduction to Robotics: Mechanics and Control (4th ed.). Pearson.

- [52] Corke, P. (2017). *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*. Springer.
- [53] LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press.